

MODULE 24

SOAP Web Service Endpoints and Request Mapping

Build, secure, and integrate contract-first SOAP web services using Spring Web Services (Spring WS) endpoints.





MODULE 24 OVERVIEW

Build, secure, and integrate contract-first SOAP web services using Spring Web Services (Spring WS) endpoints.

Many enterprises still expose SOAP services alongside REST -- this module covers contract-first design with WSDL/XSD, @Endpoint routing, and WS-Security, then ships a Spring-WS service beside the Module 23 REST API.

DURATION & LAB



4 Hours Theory

Contract-first SOAP, WSDL/XSD, @Endpoint and @PayloadRoot mapping, marshalling, faults, and WS-Security.



2 Hours Lab

Northstar CRM Spring-WS: generate a live WSDL from an XSD and implement four SOAP operations.



Scenario

A regional billing partner only integrates via SOAP/XML and needs create, get, update, and list operations.

MODULE ARC



Understand Contract-First

The XSD -- not Java -- is the source of truth; WSDL is generated dynamically from it.



Master Endpoints & Faults

@Endpoint/@PayloadRoot routing, XML marshalling, structured SOAP faults, and UsernameToken security.



Build the Lab

Implement CustomerEndpoint's four operations and verify with curl and MockWebServiceImpl.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours hands-on lab. Protocol may differ from REST, but business rules must not -- both delegate to the same CustomerService.

WHY SOAP REMAINS IMPORTANT

SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.

While REST has become popular, SOAP continues to be critical in many enterprise environments.

- **Enterprise-Grade Security** — built-in support for WS-Security standards provides message-level security, encryption, signatures, and authentication.
- **Formal Contract and Strong Typing** — WSDL and XML Schema enforce strict contracts, ensuring clarity, validation, and robust integration.
- **Reliability and Standards** — uses proven protocols (HTTP, SMTP, etc.) and standards supported by major vendors and organizations.
- **Enterprise and Legacy Integration** — ideal for integrating with legacy systems, mainframes, and heterogeneous platforms.
- **Governance and Compliance** — meets regulatory and compliance requirements in industries like banking, healthcare, and insurance.

SOAP IS STILL RELEVANT WHERE...

Secure by Design

Standards Based

Reliable Messaging

Compliance Ready

Legacy System Integration

Enterprise Adoption

KEY TAKEAWAY SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design, develop, and secure SOAP web services using Spring WS.

- **Review SOAP Fundamentals** — recall SOAP architecture, messages, WSDL, and XML Schema concepts.
- **Understand Spring Web Services** — explain the purpose, architecture, and core components of Spring WS.
- **Apply Contract-First Development** — design SOAP services using WSDL and XML Schema before writing code.
- **Build SOAP Endpoints** — create and configure endpoints using @Endpoint and @PayloadRoot.
- **Process SOAP Requests** — implement the request/response processing workflow and message lifecycle.
- **Work with XML** — perform XML marshalling and unmarshalling of SOAP payloads.
- **Handle SOAP Faults** — structure and translate business errors into standards-compliant SOAP faults.
- **Implement WS-Security** — apply WS-Security concepts, including UsernameToken, to secure SOAP services.
- **Integrate Enterprise SOAP Services** — consume and expose SOAP services in enterprise environments.
- **Follow Enterprise Best Practices** — apply best practices for reliable, secure, maintainable SOAP services.

KEY TAKEAWAY These objectives build the skills to design, build, secure, and integrate production-grade SOAP endpoints with Spring WS.

SOAP ARCHITECTURE RECAP

A brief refresher -- Module 13 already covered SOAP envelope, headers, faults, and versioning in depth.

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in a decentralized, distributed environment.

SOAP MESSAGE STRUCTURE

Part	Role
SOAP Envelope	The root element that defines the XML document as a SOAP message.
SOAP Header (Optional)	Carries metadata such as security, transactions, routing, and other context.
SOAP Body (Mandatory)	Carries the actual request or response data (the payload).
Transport Layer	Typically HTTP/HTTPS, SMTP, or JMS -- moves the SOAP message between client and server.

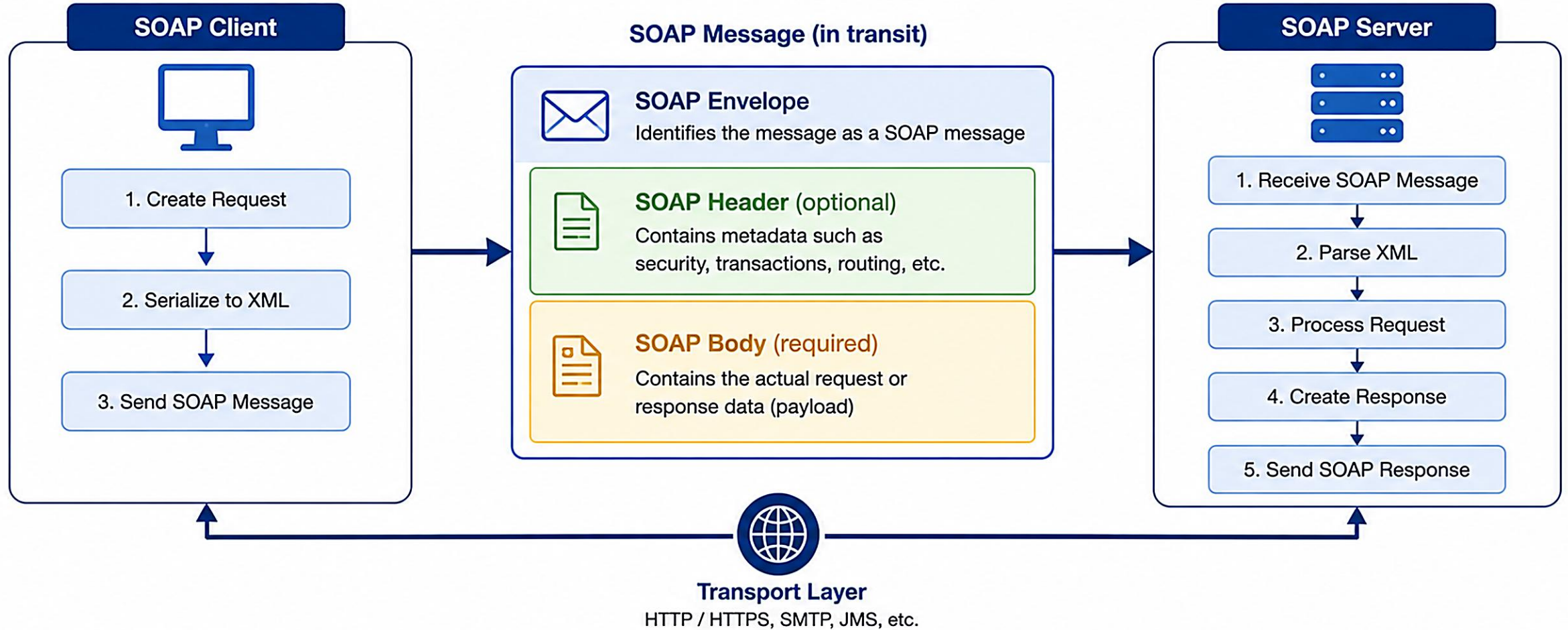
HOW IT WORKS



KEY TAKEAWAY SOAP's formal envelope, header, and body structure keeps it standards-based and enterprise-ready.

SOAP Architecture

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in the implementation of web services over a network.



Key Points

- SOAP uses XML for message format.
- Messages are transferred over standard protocols like HTTP/HTTPS.
- SOAP is platform and language independent.

WSDL REVIEW (1 OF 2)

WSDL (Web Services Description Language) is an XML-based language that describes a SOAP service and how to access it.

PURPOSE OF WSDL

- Describes what the service does, how it can be accessed, where it is located, and what data formats it uses.

KEY COMPONENTS OF WSDL

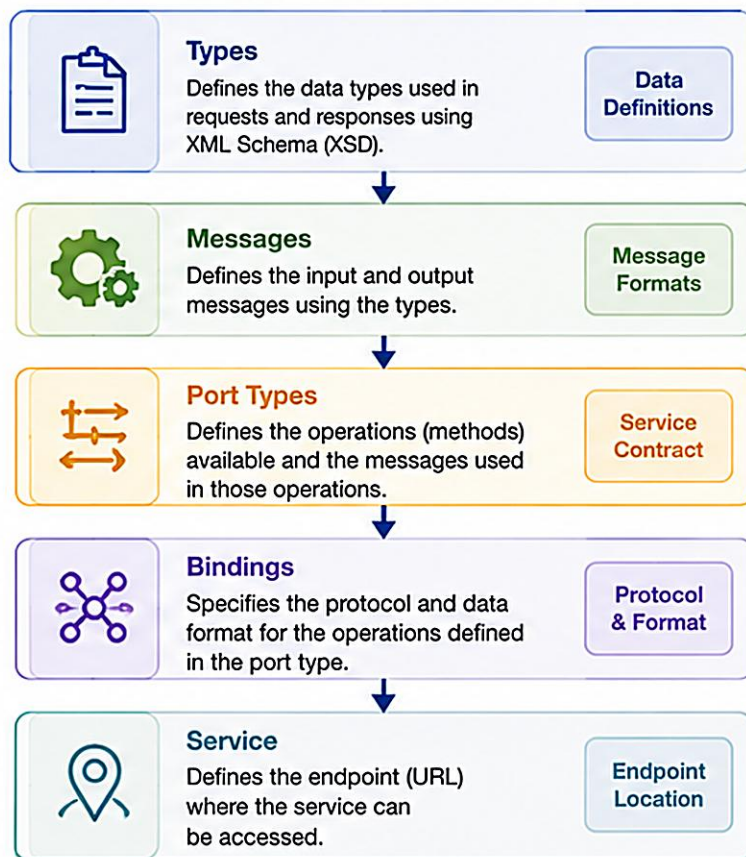
Component	Defines
types	Reusable data types used by the service, defined using XML Schema (XSD).
message	The input and output messages for the operations.
portType	The interface -- the set of operations (methods) the service exposes.
binding	The protocol (SOAP) and data format (e.g. document/literal) used to communicate.
service	The actual endpoint URL where the service can be accessed.

KEY TAKEAWAY types -> message -> portType -> binding -> service: WSDL builds from reusable data types up to a live, addressable endpoint.

WSDL (Web Services Description Language)

WSDL is an XML based contract that describes a web service. It defines what the service does, how to access it, and what the request and response messages look like.

WSDL Structure Overview



WSDL Example (Simplified)

```
WSDL
<definitions name="HelloService"
  targetNamespace="http://example.com/hello"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://example.com/hello"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

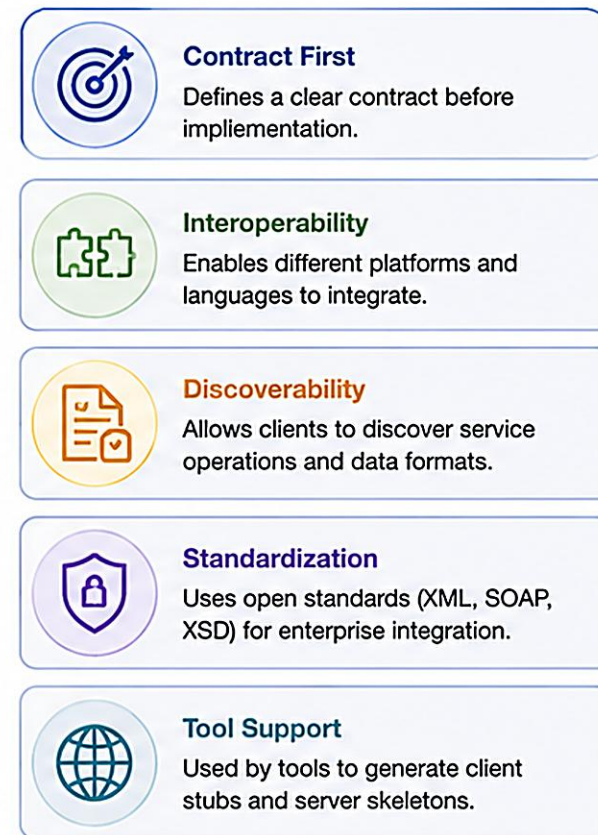
  <!-- Types: Data definitions -->
  <types>
    <xsd:schema targetNamespace="http://example.com/hello">
      <xsd:element name="sayHelloRequest">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="name" type="xsd:string"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
      <xsd:element name="sayHelloResponse">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="greeting" type="xsd:string"/>
          </xsd:sequence>
        </xsd:complexType>
      </xsd:element>
    </xsd:schema>
  </types>

  <!-- Messages -->
  <message name="SayHelloRequest">
    <part name="parameters" element="tns:sayHelloRequest"/>
  </message>
  <message name="SayHelloResponse">
    <part name="parameters" element="tns:sayHelloResponse"/>
  </message>

  <!-- Port Type (Operations) -->
  <portType name="HelloPortType">
    <operation name="sayHello">
      <input message="tns:SayHelloRequest"/>
      <output message="tns:SayHelloResponse"/>
    </operation>
  </portType>

  <!-- Binding (Protocol & Format) -->
  <binding name="HelloBinding" type="tns:HelloPortType">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="sayHello">
      <soap:operation soapAction="http://example.com/hello/sayHello"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
</definitions>
```

Key Purposes of WSDL



Key Takeaways

- WSDL is the contract of a SOAP web service.
- It defines data types, messages, operations, protocol, and endpoint.
- Clients use WSDL to understand how to interact with the service.
- Essential for enterprise, cross-platform, and loosely coupled systems.

WSDL REVIEW (2 OF 2)

A sample WSDL document, how it is used end to end, and why it matters for contract-first integration.

Sample WSDL Snippet

```
<definitions name="CustomerService" targetNamespace="http://northstar.com/crm/customer"
  xmlns:tns="http://northstar.com/crm/customer" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- types --> <types> ... </types>
  <!-- messages --> <message name="GetCustomerRequest"> ... </message>
  <!-- portType --> <portType name="CustomerServicePort"> ... </portType>
  <!-- binding --> <binding name="CustomerBinding" type="tns:CustomerServicePort">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>
  <!-- service --> <service name="CustomerService">
    <port name="CustomerPort" binding="tns:CustomerBinding">
      <soap:address location="http://localhost:8080/ws"/>
    </port>
  </service>
</definitions>
```

HOW WSDL IS USED

Provider Creates WSDL

Client Reads WSDL

Client Generates Stub

Exchange SOAP Messages

WHY WSDL MATTERS

Contract-First Dev

Loose Coupling

Interoperability

Legacy Integration

KEY TAKEAWAY DefaultWsd11Definition generates WSDL dynamically from your XSD, so it never drifts from the contract.

XML SCHEMA REVIEW (1 OF 2)

XML Schema (XSD) defines the structure, data types, and constraints for XML documents -- the foundation for strong typing in SOAP.

KEY CONCEPTS

Concept	Description
Elements	Define the structure and hierarchy of XML data.
Data Types	Define the type of data (string, int, boolean, date, etc.).
Constraints	Apply rules like minOccurs, maxOccurs, length, pattern, etc.
Complex Types	Define elements that contain other elements or attributes.
Attributes	Provide additional information about elements.

XML SCHEMA STRUCTURE

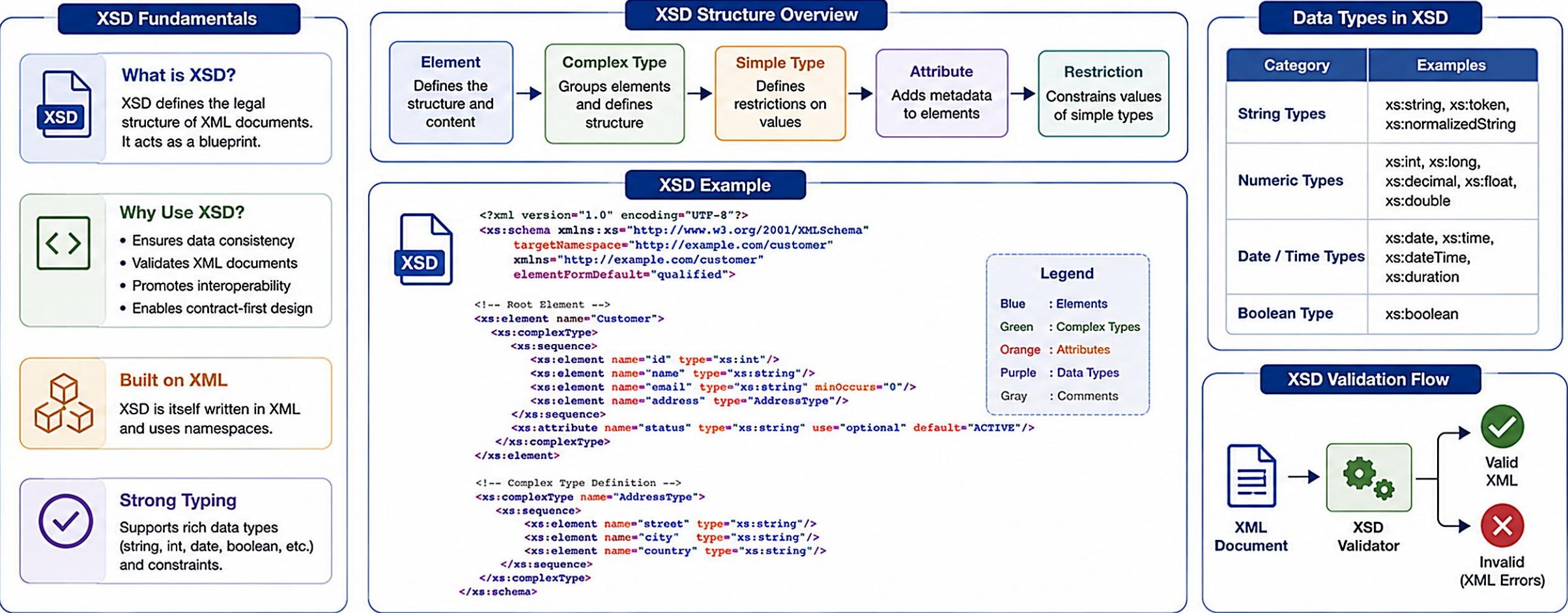


KEY TAKEAWAY XSD is reused by WSDL to define message formats for SOAP operations -- it is the single source of truth for the contract.



XML Schema (XSD)

XML Schema Definition (XSD) is an XML based language used to describe the structure, data types, and constraints of XML documents.



XML SCHEMA REVIEW (2 OF 2)

A sample XSD, its benefits, and where XML Schema is used across SOAP web services.

Sample XML Schema (XSD) -- customer.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:tns="http://northstar.com/crm/customer"
  targetNamespace="http://northstar.com/crm/customer" elementFormDefault="qualified">
  <xs:simpleType name="CustomerStatus">
    <xs:restriction base="xs:string">
      <xs:enumeration value="PROSPECT"/> <xs:enumeration value="ACTIVE"/>
      <xs:enumeration value="SUSPENDED"/> <xs:enumeration value="CLOSED"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="CustomerType">
    <xs:sequence>
      <xs:element name="customerId" type="xs:string"/>
      <xs:element name="status" type="tns:CustomerStatus"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

BENEFITS OF XML SCHEMA

Validates XML

Enforces Data Types

Improves Interoperability

Reduces Integration Errors

COMMON USE CASES

Request/Response Structures

Data Validation

JAXB Code Generation

KEY TAKEAWAY jaxb2-maven-plugin generates Java classes directly from this XSD -- the schema drives the code.

WHAT IS SPRING WEB SERVICES?

Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.

Spring Web Services (Spring WS) is a lightweight, extensible framework for building SOAP-based web services and clients in the Java ecosystem. It simplifies contract-first SOAP development using XML, WSDL, and XSD.

KEY HIGHLIGHTS



Standards Based

Built on SOAP, WSDL, XML Schema, and other open standards.



Contract-First

Service contracts are defined using WSDL and XSD before implementation.



Lightweight & Flexible

Lightweight vs. full JAX-WS; integrates easily with the Spring ecosystem.



Enterprise Ready

Supports WS-Security, logging, validation, exception handling, monitoring.

COMMON USE CASES

Enterprise System Integration

B2B / Partner Integrations

Legacy System Modernization

Compliance-Driven Solutions

Cross-Org Data Exchange

KEY TAKEAWAY Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.

SPRING WS ARCHITECTURE

Spring Web Services provides a flexible, extensible, contract-first framework for building SOAP-based web services.

Component	Responsibility
Message Dispatcher	Receives incoming SOAP requests and dispatches them to the correct endpoint.
Endpoint Adapter	Bridges the transport layer and the endpoint by invoking the business method.
@Endpoint	POJO annotated with @Endpoint containing the business logic for the service.
Marshalling / Unmarshalling	Uses JAXB (or another marshaller) to convert between Java objects and XML.
Message Factory	Creates and processes SOAP messages and manages SOAP headers and payload.
Transport Layer	Responsible for delivering messages over HTTP, SMTP, JMS, etc.

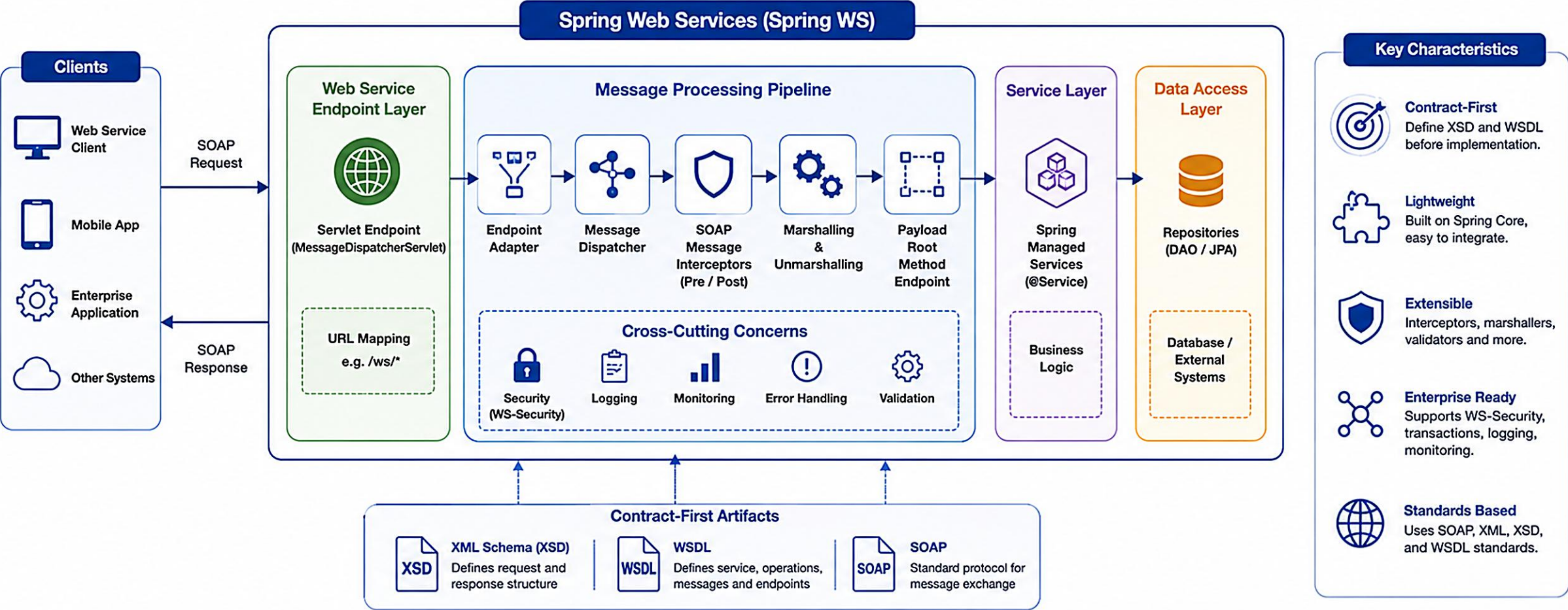
SOAP REQUEST PROCESSING FLOW



KEY TAKEAWAY Cross-cutting support (security, validation, logging) wraps every step from Dispatcher through Message Factory.

Spring WS Architecture

Spring Web Services (Spring WS) provides a lightweight framework for building SOAP-based web services using a contract-first approach.



Key Takeaways

- Spring WS provides a robust architecture for building SOAP web services.
- It follows a modular pipeline for processing SOAP requests and responses.
- Contract-first approach ensures clear service contracts and strong integration.
- Built-in support for security, monitoring, validation and error handling.

CONTRACT-FIRST DEVELOPMENT (1 OF 2)

Contract-First Development means defining the service contract (WSDL and XML Schema) before writing any code.

The contract acts as an agreement between the service provider and consumers, ensuring a clear and stable interface. After gathering requirements, design the service interface first, agree on the contract, then implement -- this reduces rework and enables parallel development.

CONTRACT-FIRST DEVELOPMENT PROCESS



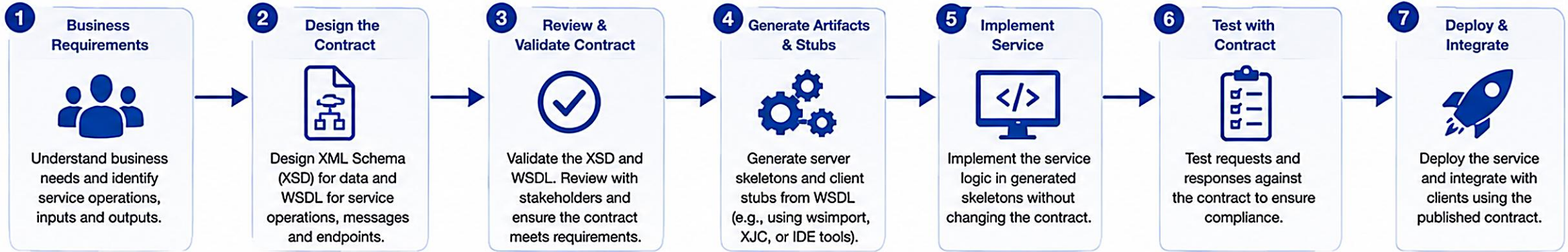
The contract (WSDL + XSD) is the single source of truth and remains stable over time -- providers and consumers can then work independently against it.

KEY TAKEAWAY In Lab 24 the XSD is authored first (Step 2); the WSDL, JAXB types, and endpoint all follow from it -- never the reverse.

Contract-First Development

Contract-First Development means designing the service contract (XSD & WSDL) before writing any code. The contract drives the implementation, ensuring clear agreements and loose coupling.

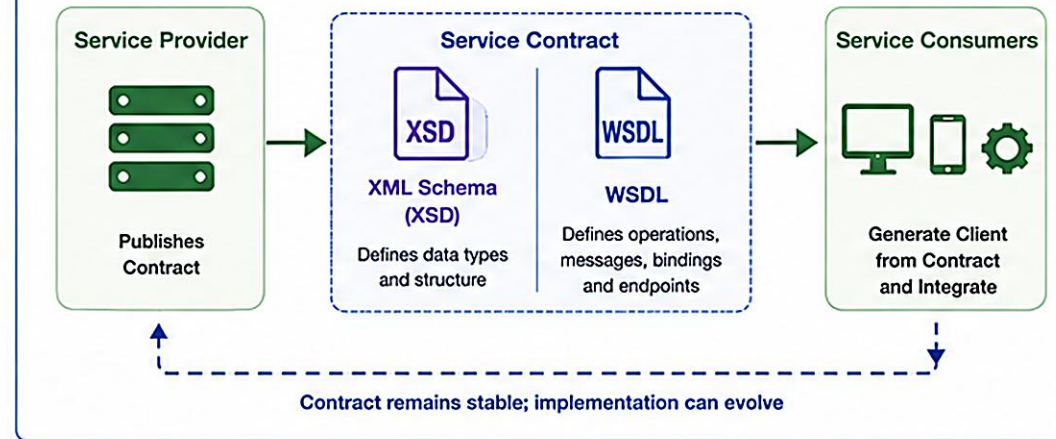
Contract-First Development Lifecycle



Benefits

- Clear Agreement**
All parties agree on the contract before implementation.
- Loose Coupling**
Changes to implementation do not affect the contract.
- Early Validation**
Issues are identified early by validating the contract.
- Parallel Development**
Service and client teams can work in parallel.
- Reusability**
Contract can be reused across multiple implementations.

Contract-First Flow



Best Practices

- Design for clarity and simplicity.
- Version your contracts.
- Maintain backward compatibility.
- Include security requirements in the contract.
- Review contract changes with all stakeholders.



Key Takeaways

- Contract-First ensures the contract drives the implementation.
- XSD and WSDL are created, validated and agreed before coding.
- It improves quality, reduces rework and enables better collaboration.
- Changes are managed through versioned contracts, not breaking implementations.

CONTRACT-FIRST DEVELOPMENT (2 OF 2)

Key artifacts, benefits, and best practices for contract-first SOAP development.

KEY ARTIFACTS

- **XSD (XML Schema)** — defines the structure, data types, and constraints of request/response messages.
- **WSDL** — describes the service, operations, message formats, and how to access it.
- **Service Contract** — the agreement between provider and consumer that should not change frequently.
- **Generated Artifacts** — server stubs, client proxies, and data bindings (via wsimport, xjc, or JAXB).

BENEFITS

Clear Agreement

Parallel Development

Reduced Rework

Consistency

Maintainability

BEST PRACTICES

- Invest time in designing a robust contract with clear documentation.
- Version your contracts (e.g., v1, v2) and maintain backward compatibility.
- Automate contract validation in the build pipeline.

KEY TAKEAWAY A well-designed, versioned contract is the single source of truth for every consumer.

TRY IT YOURSELF — EXERCISE 1: CONTRACT-FIRST RECALL

Reinforces: the XSD -> JAXB -> WSDL contract-first order and the Lab 13 link you just covered.

STEP / WHAT TO DO

Step	What To Do
Goal	Explain why the partner XSD -- not Java classes -- owns the SOAP contract.
File	notes/contract-first.md (in examples/module-24-exercises/).
Order	Align your explanation with the XSD -> JAXB -> WSDL order: XSD is the source of truth.
Lab 13 link	Note that Lab 24 implements Lab 13's customer operations over Spring-WS.
Deliverable	Contract-first rule and Lab 13 link documented; state you will not author the full XSD yet.
Reference	exercises/exercise-01-contract-first-recall.md

Contract-First Flow (restate this in your own words in notes/contract-first.md)

```
customer.xsd (hand-authored -- the single source of truth)
  | mvn generate-sources (jaxb2-maven-plugin)           v DefaultWsd111Definition
  v                                                     v
Generated JAXB types (derived, never hand-edited)    Dynamic WSDL /ws/customer.wsdl (never hand-written)
```

TRY IT NOW Complete Exercise 1 before continuing -- see exercises/exercise-01-contract-first-recall.md.

ENDPOINT ARCHITECTURE

In Spring Web Services, an endpoint is the server-side component that receives, processes, and responds to SOAP requests.

WHAT IS AN ENDPOINT?

- **Business Function** — an endpoint represents a specific business function exposed as a SOAP web service operation.
- **Annotation-Mapped** — handles incoming SOAP requests and maps them to Java methods using annotations.
- **Runtime-Managed** — operates within the Spring WS infrastructure for message handling, marshalling, and security.
- **Focused & Reusable** — kept focused on request handling and easy to maintain.

SPRING WS ENDPOINT RUNTIME



BENEFITS OF THIS ARCHITECTURE

Loose Coupling

Scalability

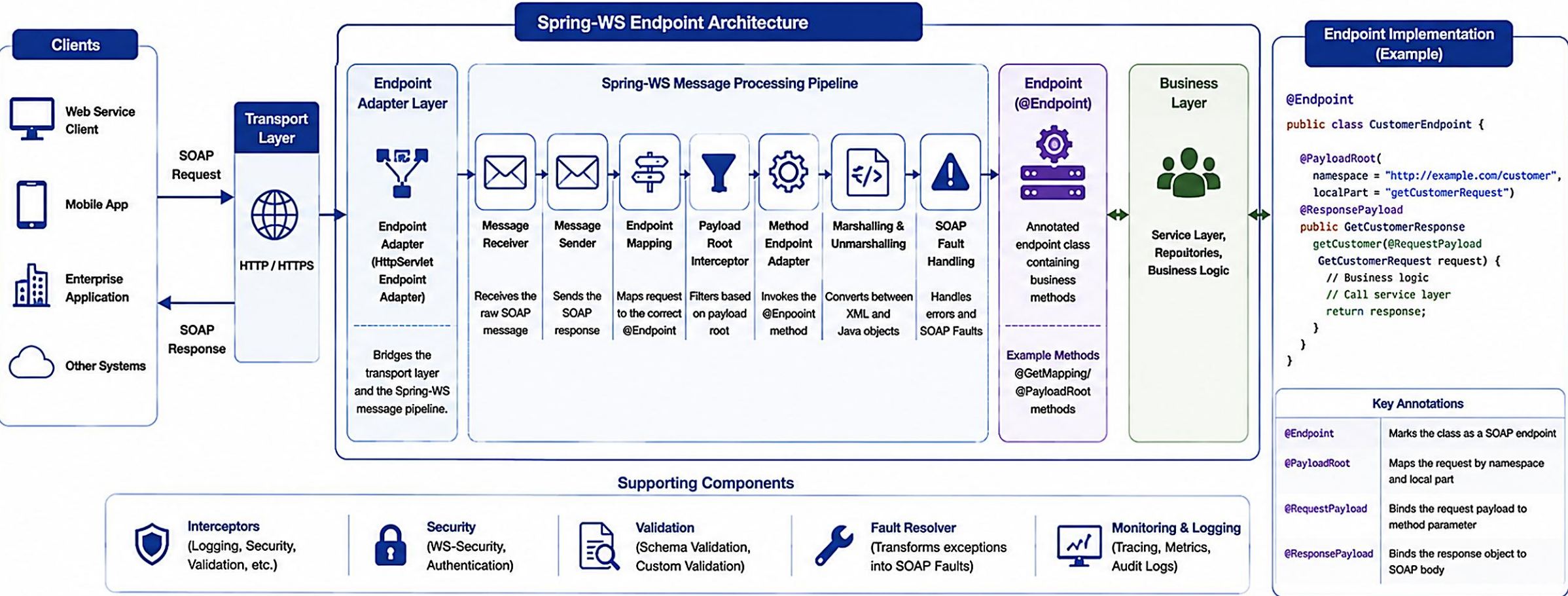
Maintainability

Standards Compliant

KEY TAKEAWAY Endpoints are decoupled from clients and from the transport layer -- add or extend them independently as the contract grows.

Endpoint Architecture

In Spring-WS, an **Endpoint** is the component that receives SOAP requests, processes them, and returns SOAP responses.



★

Key Takeaways

✓ Endpoints are the core entry point for handling SOAP requests in Spring-WS.

✓ The message pipeline processes, transforms, and routes SOAP messages.

✓ @Endpoint classes contain business methods mapped via @PayloadRoot.

✓ Interceptors, security, validation, and fault handling ensure enterprise-grade services.

✓ Clean separation: Transport → Pipeline → Endpoint → Business Layer.

@ENDPOINT ANNOTATION (1 OF 2)

The @Endpoint annotation marks a class as a SOAP endpoint -- Spring WS detects it and registers it as a web service endpoint.

WHAT IS @ENDPOINT?

- **Class-level annotation** — declares that a class contains methods that handle incoming SOAP requests.
- **Auto-registered** — allows Spring to scan and register the endpoint automatically.
- **Method mapping** — methods within the class are mapped to SOAP operations using @PayloadRoot.
- **Spring-native** — supports dependency injection and other Spring features.

HOW @ENDPOINT WORKS



KEY TAKEAWAY @Endpoint promotes clean separation of concerns -- delegate business logic to service classes and keep endpoints stateless.



@Endpoint Annotation in Spring WS

Marks a class as a Web Service endpoint to handle SOAP requests



The **@Endpoint** annotation identifies a class as a Spring Web Services endpoint. Methods inside this class handle incoming SOAP requests.

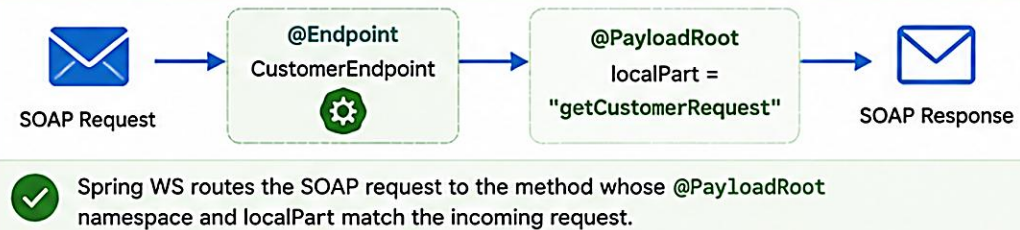
1. @Endpoint – Example

```
import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;
import javax.xml.transform.Source;

@Endpoint
public class CustomerEndpoint {
    private static final String NAMESPACE_URI = "http://example.com/customer";

    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "getCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(@RequestPayload GetCustomerRequest request) {
        // Business logic to fetch customer
        // Build and return response
        return buildResponse(request);
    }
}
```

2. Request Mapping with @Endpoint



3. Key Points



Identifies a class as a web service endpoint.



Works with **@PayloadRoot** to map SOAP requests to methods.



Methods return **@ResponsePayload** to send SOAP response.

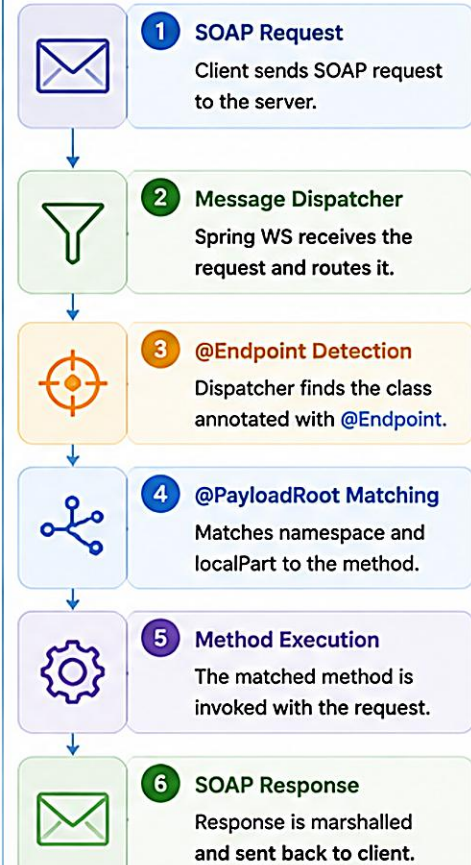


Promotes clean separation of concerns in service layer.



Enables contract-first development with WSDL and XSD.

4. Endpoint Processing Flow



5. Configuration

```
<sws:annotation-driven /> <!-- Enables annotation-based endpoints -->
```

```
<bean class="org.springframework.ws.transport.http.MessageDispatcherServlet">
    <property name="transformWsdLocations" value="true"/>
</bean>
```

@ENDPOINT ANNOTATION (2 OF 2)

A working @Endpoint example, its annotation details, and best practices for endpoint classes.

CustomerEndpoint.java

```
@Endpoint
public class CustomerEndpoint {
    private static final String NS = "http://northstar.com/crm/customer";

    private final CustomerService customerService;

    @PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(
        @RequestPayload GetCustomerRequest request) {
        // delegate to the same service REST uses
        Customer c = customerService.get(request.getCustomerId());
        return CustomerSoapMapper.toSoap(c);
    }
}
```

ANNOTATION DETAILS

Field	Value
Package	org.springframework.ws.server.endpoint.annotation
Target	TYPE (Class)
Retention	RUNTIME
Purpose	Marks a class as a SOAP endpoint

BEST PRACTICES

- Use meaningful class names related to the domain.
- Delegate business logic to service classes -- never re-implement rules in the endpoint.
- Keep endpoints stateless whenever possible.

KEY TAKEAWAY CustomerEndpoint stays a thin protocol adapter -- it delegates so SOAP and REST never fork business rules.

REQUEST MAPPING WITH @PAYLOADROOT

@PayloadRoot maps an incoming SOAP request to a specific endpoint method based on the XML payload's namespace and local name.

Attribute	Meaning
namespace	Namespace URI of the root element in the request payload.
localPart	Name of the root element to be mapped (e.g. getCustomerRequest).

Example

```
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
@ResponsePayload
public GetCustomerResponse getCustomer(
    @RequestPayload GetCustomerRequest request) { ... }
```

HOW REQUEST MAPPING WORKS



- Simple & Declarative
- Supports Contract-First
- Works with WSDL/XSD
- Keeps Methods Focused

KEY TAKEAWAY Use constants for the namespace URI -- four @PayloadRoot methods, one well-named method per operation.

TRY IT YOURSELF — EXERCISE 2: SOAP OPERATION MAP

Reinforces: mapping SOAP operations to shared CustomerService methods via @PayloadRoot you just saw.

STEP / WHAT TO DO

Step	What To Do
Goal	Map four customer SOAP operations to shared CustomerService methods.
File	notes/soap-ops.md (in examples/module-24-exercises/).
Map	CreateCustomer -> create; GetCustomer -> get by id; UpdateCustomerStatus -> status transition; ListCustomers -> list/filter.
Shared service rule	Write that REST and SOAP must share CustomerService so business rules never fork.
Deliverable	Operation map + shared-service rule; list fixtures CUS-1001, CUS-1002, CUS-9999, correlation lab24-001.
Reference	exercises/exercise-02-operation-map.md

The Map You Are Writing (localPart -> CustomerService method)

```
@PayloadRoot(namespace = NS, localPart = "createCustomerRequest")    -> customerService.create(...)
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")        -> customerService.get(...)
@PayloadRoot(namespace = NS, localPart = "updateCustomerStatusRequest") -> customerService.updateStatus(...)
@PayloadRoot(namespace = NS, localPart = "listCustomersRequest")      -> customerService.list(...)
```

TRY IT NOW Complete Exercise 2 before continuing -- see exercises/exercise-02-operation-map.md.

TRY IT YOURSELF — EXERCISE 3: PAYLOADROOT SKELETON (STARTER)

Reinforces: @Endpoint / @PayloadRoot delegation you just saw -- a fill-in-the-blank starter, not built from scratch.

notes/CustomerEndpointSketch.java (STARTER -- replace every blank)

```
@____
public class CustomerEndpoint {
    private final CustomerService service;

    public CustomerEndpoint(CustomerService service) {
        this.service = service;
    }

    @PayloadRoot(namespace = "http://northstar.example/customer",
        localPart = "____")
    @ResponsePayload
    public GetCustomerResponse get(
        @RequestPayload GetCustomerRequest request) {
        // TODO: call service.get(id) then map to JAXB response
        var customer = service.____(request.getCustomerId());
        return CustomerSoapMapper.toGetResponse(customer);
    }
}
```

STEP / WHAT TO DO

Step	What To Do
Goal	Complete a pseudocode endpoint skeleton that delegates to CustomerService.
Starter	TODO / fill-in-the-blank; replace every ____ and // TODO -- blanks do not compile.
Hints	Class annotation @Endpoint; localPart GetCustomerRequest; method service.get(...).
Self-check	No business rules live inside the endpoint beyond mapping/delegation.
Deliverable	All three blanks filled; delegation rule clear.
Reference	exercises/exercise-03-payloadroot-skeleton.md

TRY IT NOW Complete Exercise 3 (fill in every blank) before continuing -- see exercises/exercise-03-payloadroot-skeleton.md.

REQUEST PROCESSING WORKFLOW

Spring WS processes a SOAP request through well-defined steps, from the transport layer to the endpoint method and back.



KEY COMPONENTS INVOLVED

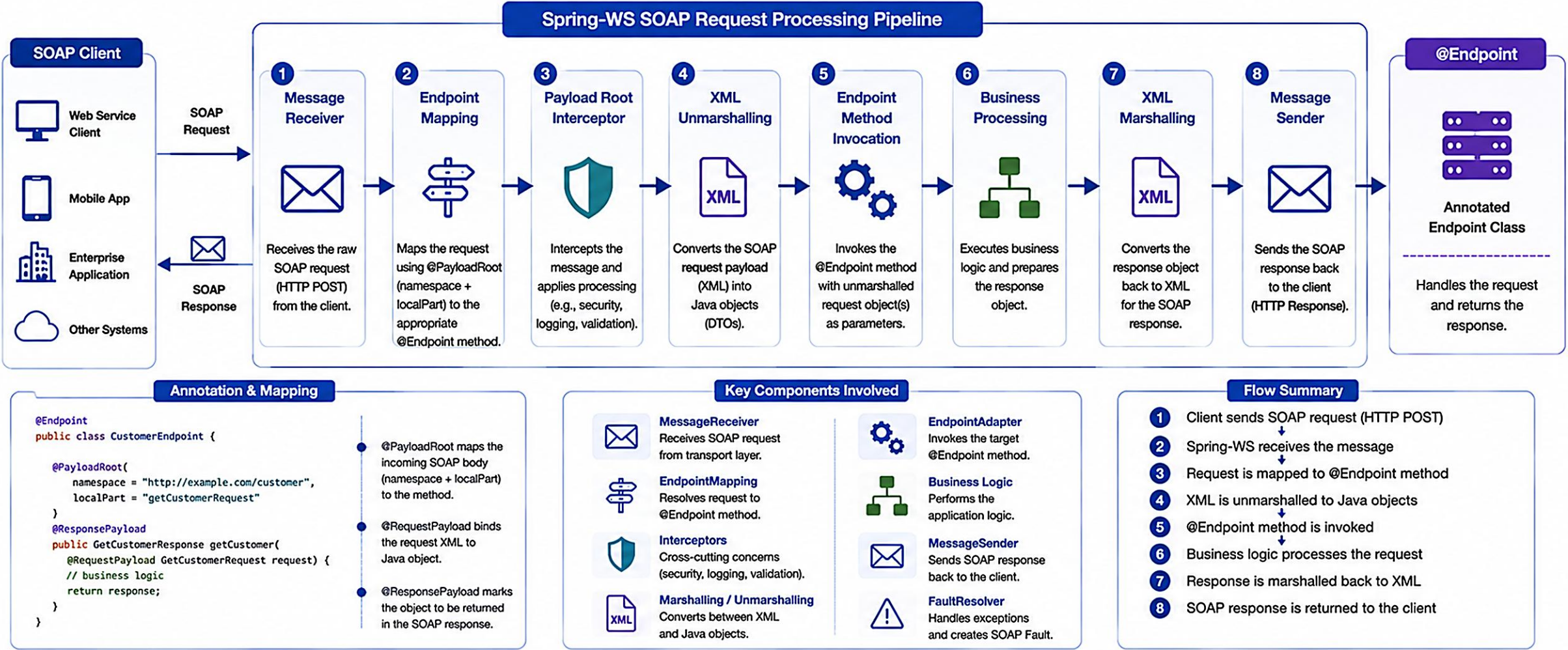
- **MessageDispatcher** — central component that routes messages to the correct endpoint.
- **@PayloadRoot** — maps the XML root element (namespace + localPart) to the handler method.
- **Interceptors (Optional)** — used for logging, security, and validation at multiple points.

Step	What Happens
1-3. Receive & Dispatch	SOAP request arrives; MessageReceiver builds a MessageContext; MessageDispatcher reads the payload.
4-6. Map, Unmarshal & Invoke	@PayloadRoot matches; payload unmarshalled to a Java object; handler method invoked.
7-8. Marshal & Send	Return value marshalled to XML; SOAP response sent back to the client.

KEY TAKEAWAY A clear, predictable request flow with loose coupling between client and service -- and easy to extend with interceptors.

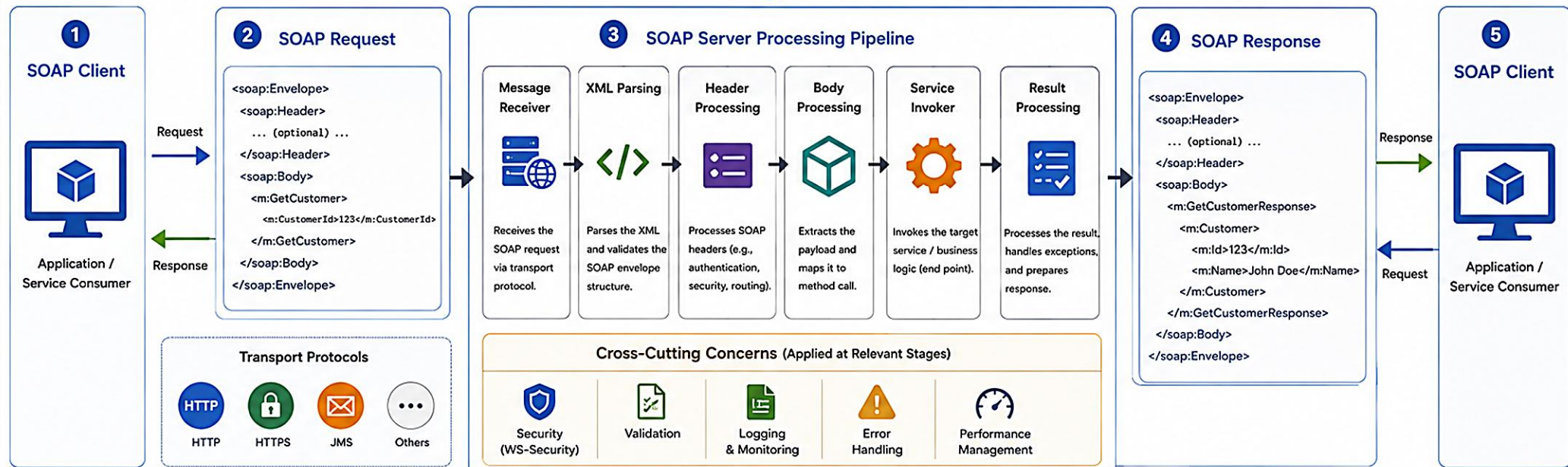
SOAP Request Processing Workflow

How a SOAP request flows through Spring-WS from the client to the @Endpoint method.

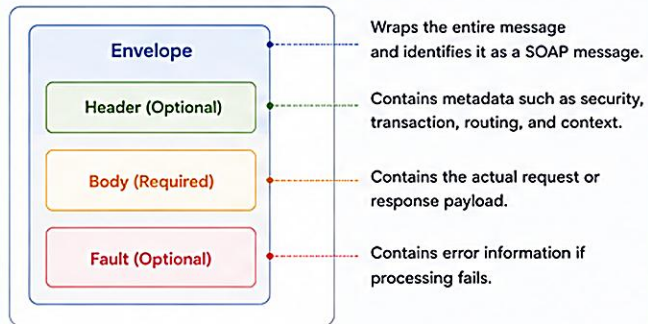


SOAP Request Processing Architecture

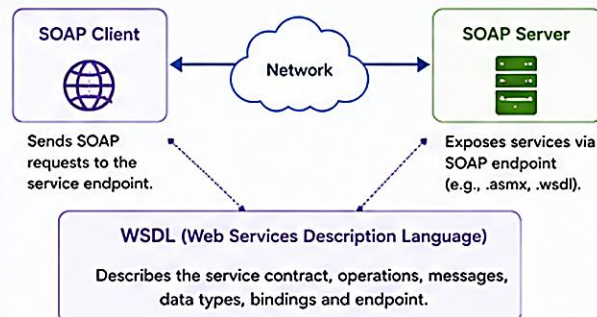
End-to-end flow of a SOAP request from client to server and back



SOAP Message Structure



Typical SOAP Endpoints (Deployment Style)



Key Benefits of SOAP Architecture

- Standard & Interoperable**: W3C standard ensures platform and language independence.
- Secure**: Supports WS-Security, encryption, digital signatures.
- Contract-First**: WSDL defines a strict contract between client and server.
- Reliable**: Built-in error handling with SOAP Fault and robust messaging.
- Enterprise Ready**: Ideal for large-scale, mission-critical enterprise systems.

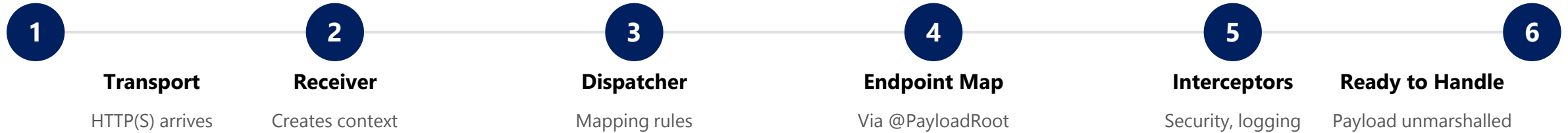
i SOAP + XML + Standards = Interoperable, Secure, and Reliable Enterprise Integration

→ Request Flow → Response Flow

💡 SOAP is transport neutral and can work over HTTP, HTTPS, JMS and other protocols.

SOAP REQUEST LIFECYCLE

The journey of a request from the client to the Spring WS endpoint, until it is ready for the handler method.



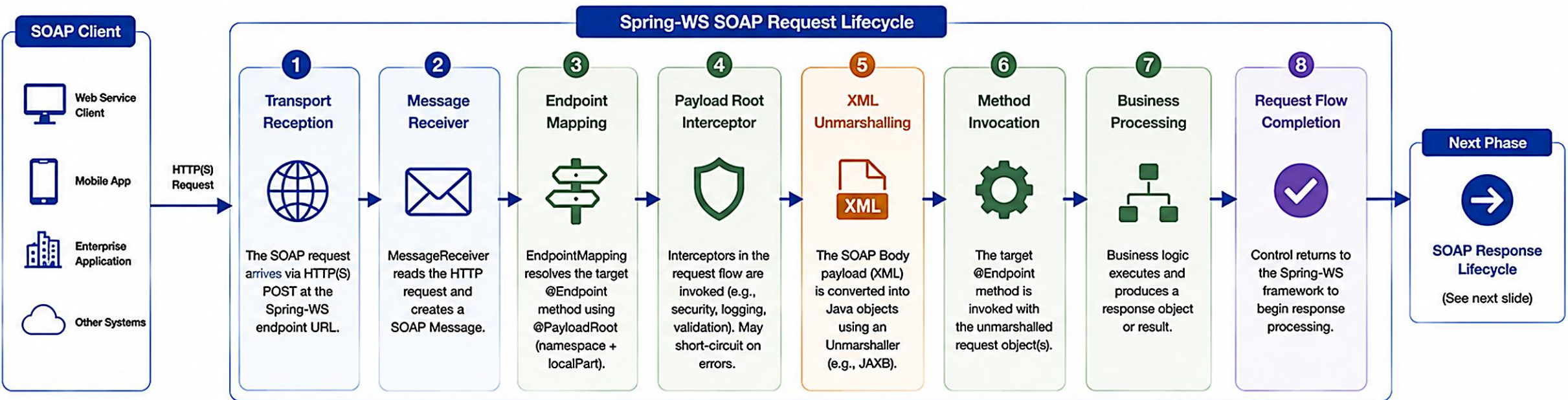
WHAT HAPPENS IN THIS LIFECYCLE?

- The HTTP request is accepted and the SOAP envelope is parsed.
- The correct endpoint is resolved using @PayloadRoot mapping information.
- Cross-cutting concerns (security, logging, validation) are applied via interceptors.
- The payload is converted from XML to Java objects before the handler runs.

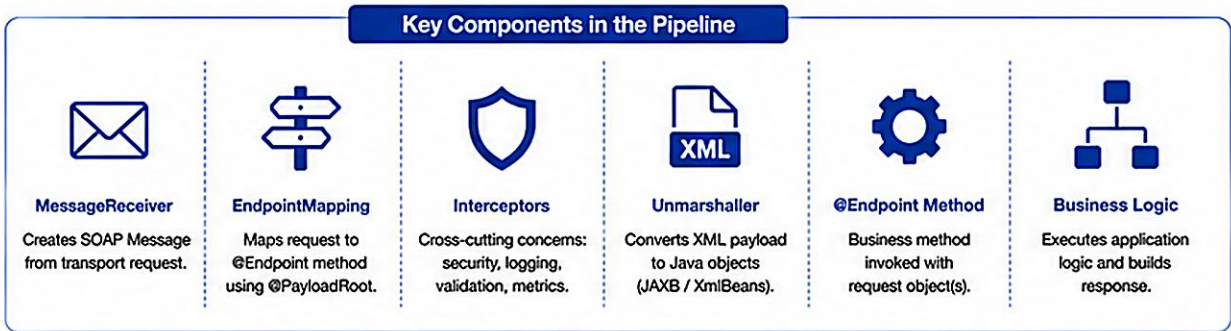
KEY TAKEAWAY These steps occur for every incoming SOAP request before the business logic is executed.

SOAP Request Lifecycle

How a SOAP request is processed inside Spring-WS from transport entry to business method invocation.



Participants	
	Client
Sends the SOAP request	
	Spring-WS Framework
Handles transport, routing, and processing	
	@Endpoint
Contains business methods	
	Backend Systems
Used by business logic if needed	



- Lifecycle Summary
- 1

Transport layer receives the SOAP request.
- 2

SOAP Message is created.
- 3

Target @Endpoint method is resolved.
- 4

Interceptors in the request flow execute.
- 5

XML payload is unmarshalled to Java objects.
- 6

@Endpoint method is invoked.
- 7

Business logic processes the request.
- 8

Control returns to framework for response phase.



Key Takeaways



The request lifecycle transforms a SOAP message into a Java method invocation.



Interceptors enable security, validation, logging, and other cross-cutting concerns.



Unmarshalling converts XML payloads into strongly typed Java objects.



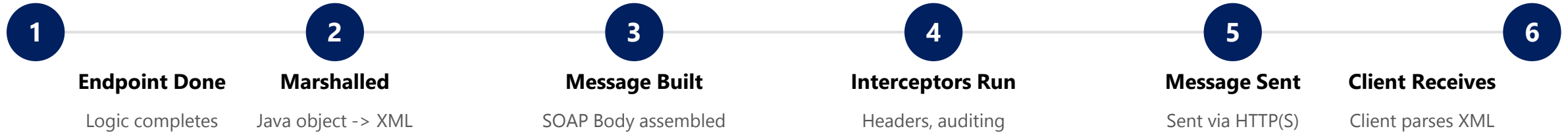
Clear pipeline stages ensure maintainability, extensibility, and observability.

31

© 2026 by Innovation In Software Corporation

SOAP RESPONSE LIFECYCLE

The journey of a response from the endpoint method back to the client, until it is received and processed.



WHAT IF SOMETHING GOES WRONG?

If an exception is thrown in the endpoint method, an appropriate SOAP Fault is created during this lifecycle and returned to the client instead of a normal response -- see the next section.

Consistent Handling

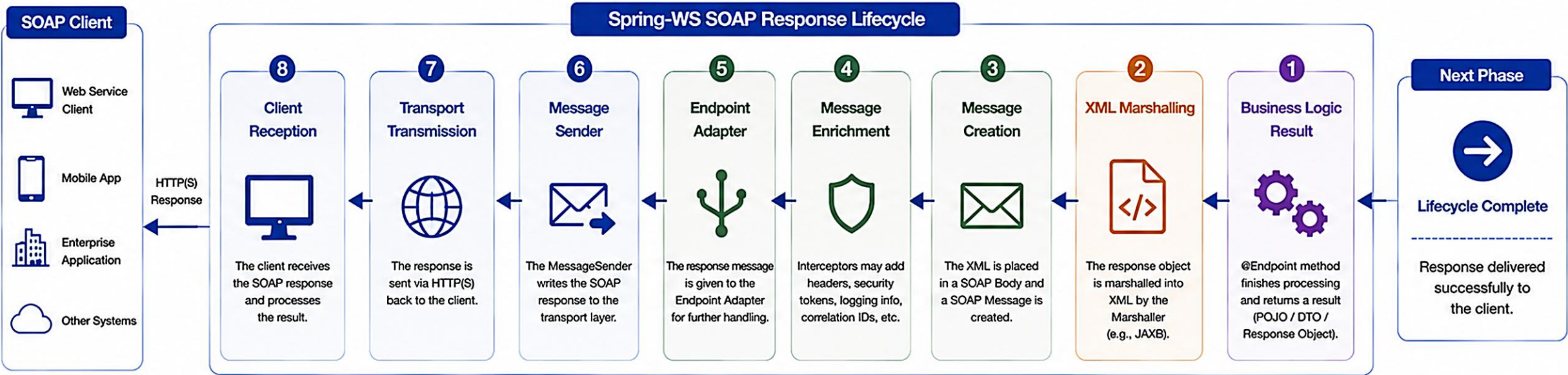
Supports Cross-Cutting Concerns

Easier Troubleshooting

KEY TAKEAWAY The response lifecycle mirrors the request lifecycle -- and it's where a thrown exception becomes a SOAP Fault.

SOAP Response Lifecycle

How a SOAP response is generated in Spring-WS and returned to the client.



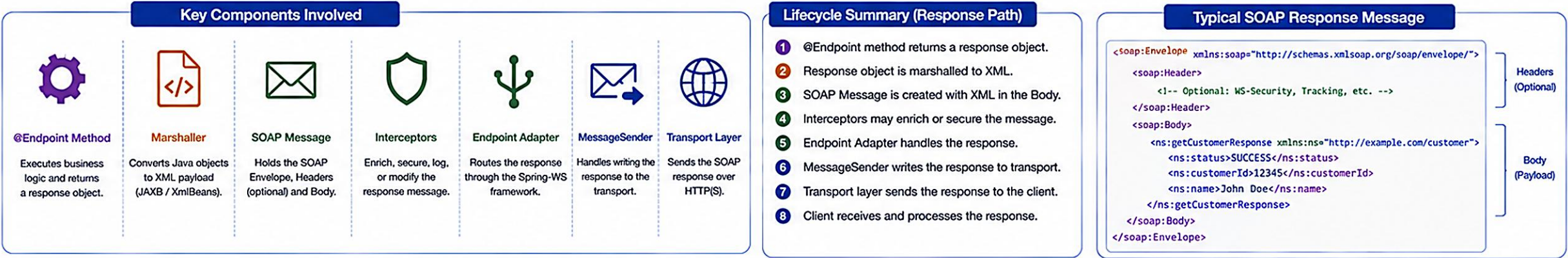
Legend

Application / Business

Marshalling

Processing

Transport



★

Key Takeaways

The response travels back through the same pipeline in reverse.

Marshalling converts Java objects to XML for the SOAP Body.

Interceptors ensure security, logging and consistency.

A well-structured lifecycle ensures reliable and standardized responses.

33

XML MARSHALLING

XML Marshalling is the process of converting Java objects into XML so they can be included in a SOAP message.



Java Object

```
Customer customer = new Customer();
customer.setId("CUS-1001");
customer.setName("Amina Khan");
customer.setEmail("amina@northstar.com");
```

Marshaled XML (Output)

```
<customer>
  <id>CUS-1001</id>
  <name>Amina Khan</name>
  <email>amina@northstar.com</email>
</customer>
```

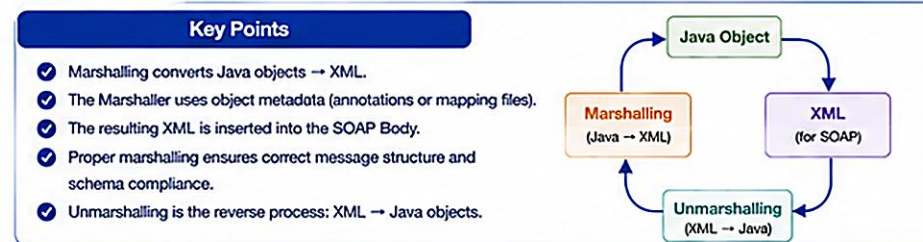
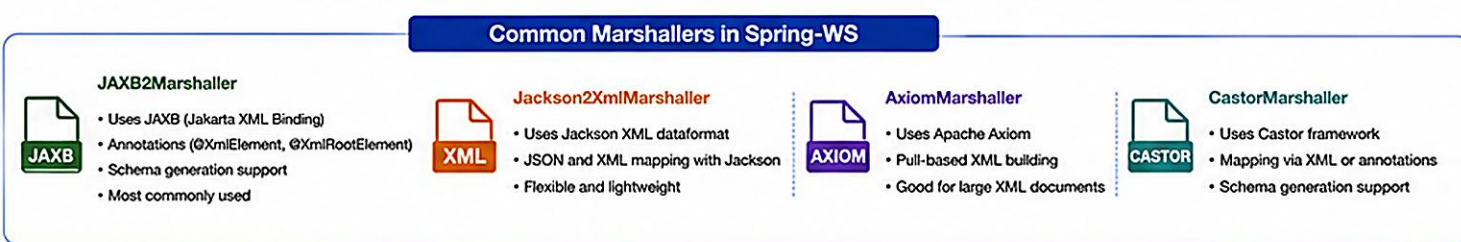
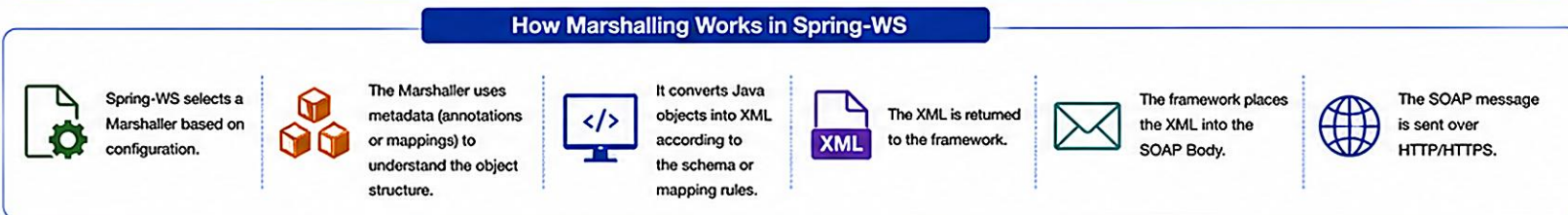
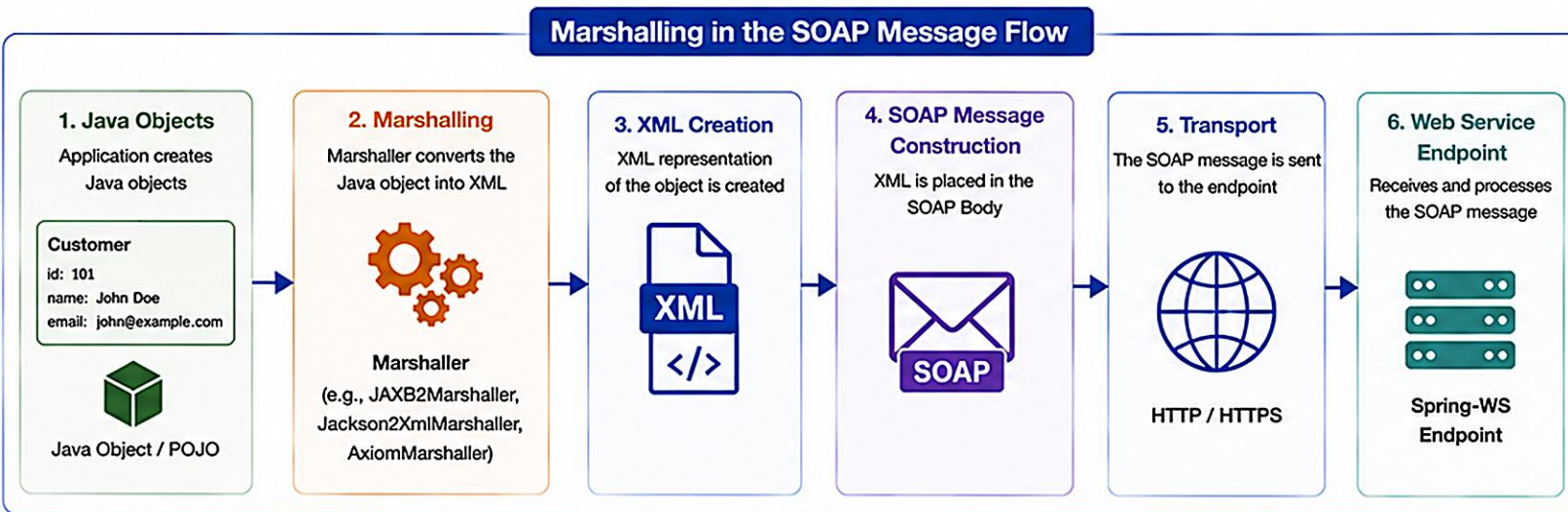
KEY POINTS

- Uses a Marshaller (typically JAXB / Jakarta XML Binding) to convert Java objects to XML.
- Follows the structure defined in the XSD or JAXB annotations -- always validate against the schema first.

KEY TAKEAWAY Marshalling ensures your Java data is correctly transformed into XML that follows the SOAP contract.

Marshalling in SOAP (Spring-WS)

Marshalling is the process of converting Java objects into XML so they can be included in the SOAP message request or response.



Example

Java Object (POJO)

```
public class Customer {  
    private Long id;  
    private String name;  
    private String email;  
    // getters and setters  
}
```

Marshaled XML

```
<ns:Customer xmlns:ns="http://example.com/schema">  
  <ns:id>101</ns:id>  
  <ns:name>John Doe</ns:name>  
  <ns:email>john@example.com</ns:email>  
</ns:Customer>
```

SOAP Message (Request) – With Marshalled XML

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"  
  xmlns:ns="http://example.com/schema">  
  <soapenv:Header/>  
  <soapenv:Body>  
    <ns:getCustomerRequest>  
      <ns:Customer>  
        <ns:id>101</ns:id>  
        <ns:name>John Doe</ns:name>  
        <ns:email>john@example.com</ns:email>  
      </ns:Customer>  
    </ns:getCustomerRequest>  
  </soapenv:Body>  
</soapenv:Envelope>
```



Key Takeaways



Marshalling is essential for creating SOAP messages from Java objects.



Spring-WS provides multiple marshallers to choose from.



Using JAXB2Marshaller with annotations is the most widely used approach.



Correct marshalling ensures interoperability, validation, and standards compliance.



Marshalling happens automatically in the request/response pipeline based on configuration.

XML UNMARSHALLING

XML Unmarshalling converts XML from a SOAP request into Java objects so the application can process the data.



SOAP Request XML

```
<getCustomerRequest>
  <customerId>CUS-1001</customerId>
</getCustomerRequest>
```

Java Object (Output)

```
GetCustomerRequest request =
    new GetCustomerRequest();
request.setCustomerId("CUS-1001");
// passed to the @RequestPayload parameter
```

KEY POINTS

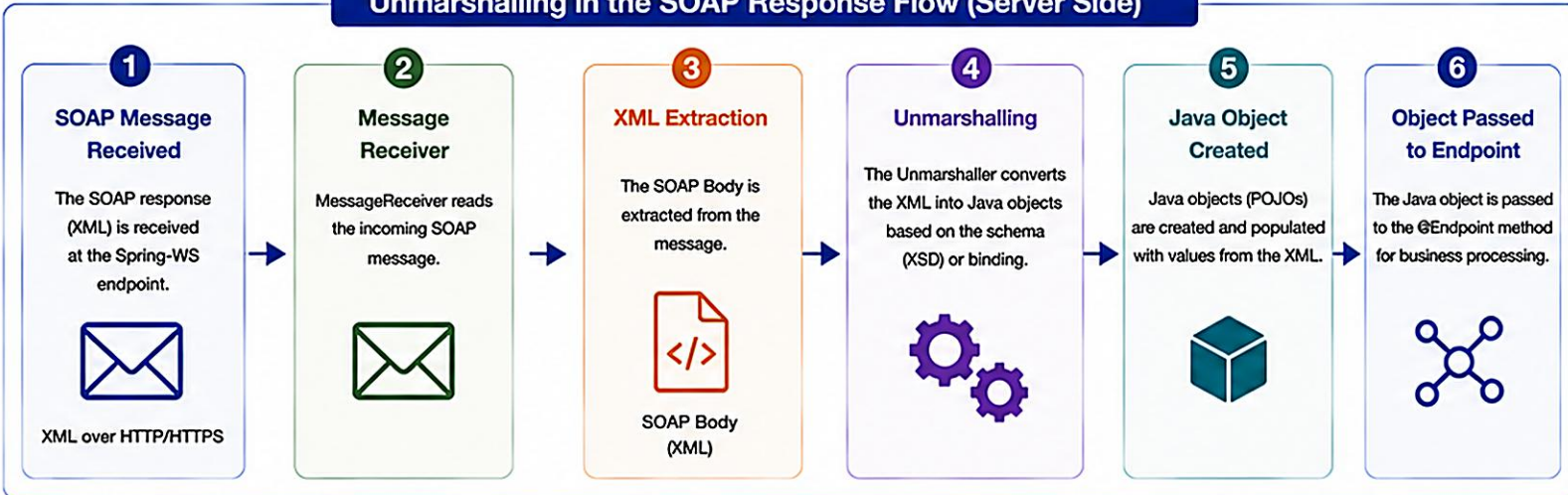
- Extracts data from the SOAP request and validates the XML structure using XSD (if configured).
- Populates strongly-typed Java objects (POJOs) for business processing -- no manual XML parsing.

KEY TAKEAWAY Unmarshalling transforms XML into usable Java objects for clean, type-safe business logic.

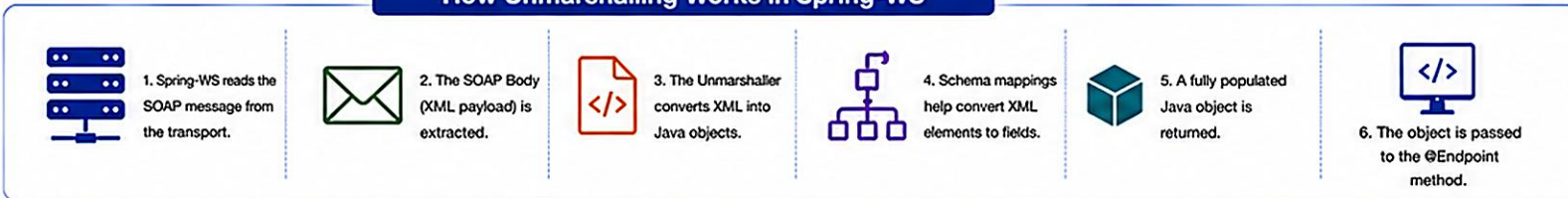
Unmarshalling in SOAP (Spring-WS)

Unmarshalling is the process of converting XML from a SOAP message into Java objects so that the application can process the data.

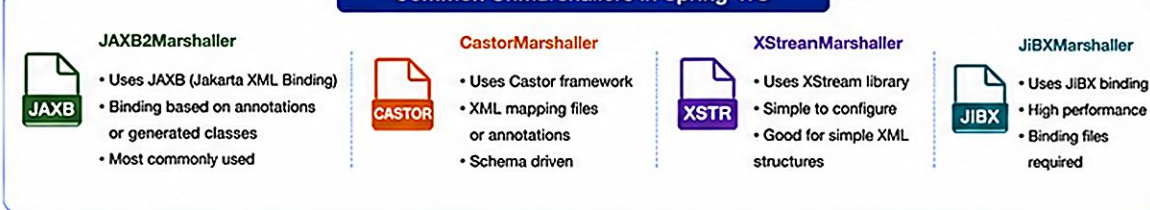
Unmarshalling in the SOAP Response Flow (Server Side)



How Unmarshalling Works in Spring-WS

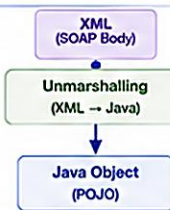


Common Unmarshallers in Spring-WS



Key Points

- ✓ Unmarshalling converts XML → Java objects.
- ✓ It is the reverse of Marshalling.
- ✓ The Unmarshaller uses schema or bindings to map XML to Java.
- ✓ Correct schema and mappings ensure accurate object creation.
- ✓ Unmarshalling occurs before business logic execution in @Endpoint methods.



Example: Incoming SOAP Response (XML)

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:ns="http://example.com/schema">
  <soapenv:Header/>
  <soapenv:Body>
    <ns:getCustomerResponse>
      <ns:status>SUCCESS</ns:status>
      <ns:customer>
        <ns:id>101</ns:id>
        <ns:name>John Doe</ns:name>
        <ns:email>john@example.com</ns:email>
      </ns:customer>
    </ns:getCustomerResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

XML

Java Object After Unmarshalling (POJO)

```
public class CustomerResponse {
    private String status;
    private Customer customer;
    // getters and setters
}

public class Customer {
    private Long id;
    private String name;
    private String email;
    // getters and setters
}
```

Unmarshalling Configuration in Spring-WS

```
<bean id="unmarshaller" class="org.springframework.xml.jaxb.Jaxb2Marshaller">
  <property name="contextPath" value="com.example.schema"/>
</bean>
```

The Unmarshaller uses the schema (XSD) or JAXB bindings to convert XML into Java objects.



Key Takeaways



Unmarshalling is essential for reading and processing SOAP responses.



It transforms XML payloads into Java objects automatically.



Schema or binding files define how XML maps to Java.



Accurate mappings improve reliability, maintainability, and reduce errors.



Spring-WS provides multiple Unmarshallers to choose from based on needs.

SOAP FAULT STRUCTURE AND ERROR HANDLING

When an endpoint throws an exception, Spring WS converts it into a structured SOAP Fault instead of a raw stack trace.

ANATOMY OF A SOAP FAULT

- **faultcode** — a qualified code such as Client or Server, identifying who is at fault.
- **faultstring** — a short, human-readable description of the error (never a stack trace).
- **faultactor** — identifies which node in the message path caused the fault (optional).
- **detail** — optional structured, application-specific error information.

Fault Code	Meaning
Client	Caller sent a bad/invalid request (e.g. not-found, duplicate).
Server	Something failed on the server side; generic message, no internals.
MustUnderstand	A required SOAP header element was not understood.
VersionMismatch	The SOAP envelope version is not supported.

SoapFaultMappingExceptionResolver

```
@Bean
SoapFaultMappingExceptionResolver exceptionResolver() {
    var resolver = new SoapFaultMappingExceptionResolver();
    var mappings = new Properties();
    mappings.setProperty(
        "com.northstar.crm.exception.CustomerNotFoundException",
        "CLIENT, Customer not found");
    resolver.setExceptionMappings(mappings);
    return resolver;
}
```

Fault Response (CUS-9999, not found)

```
<SOAP-ENV:Fault>
  <faultcode>SOAP-ENV:Client</faultcode>
  <faultstring>Customer not found</faultstring>
</SOAP-ENV:Fault>
```

Never let faultstring or detail leak stack traces or secrets.

KEY TAKEAWAY Map business exceptions to CLIENT faults with clear messages; default everything else to SERVER -- no stack traces, ever.

TRY IT YOURSELF — EXERCISE 4: SOAP FAULT VS. REST ERROR

Reinforces: mapping business exceptions to SOAP faults without leaking stacks you just covered.

STEP / WHAT TO DO

Step	What To Do
Goal	Document how business exceptions become SOAP faults without leaking stacks.
File	notes/fault-vs-rest.md (in examples/module-24-exercises/).
Contrast table	Columns Concern SOAP REST; rows: not-found, validation, missing UsernameToken.
Answer sketch	Not-found -> SOAP fault vs HTTP 404 JSON; missing token -> security fault vs 401 on REST.
Deliverable	Contrast table + no-stack-trace rule; note Lab 16 exceptions feed Lab 24 fault mapping.
Reference	exercises/exercise-04-fault-vs-rest.md

Same Failure, Two Protocols (what your contrast table is documenting)

```
SOAP: <SOAP-ENV:Fault><faultcode>SOAP-ENV:Client</faultcode><faultstring>Customer not found</faultstring></SOAP-ENV:Fault>
REST: HTTP 404 { "error": "NOT_FOUND", "message": "Customer not found" }
Rule: faultstring/detail and REST error bodies never include stack traces, class names, or secrets.
```

TRY IT NOW Complete Exercise 4 before continuing -- see exercises/exercise-04-fault-vs-rest.md.

SOAP SECURITY OVERVIEW

SOAP provides several standards and mechanisms (WS-Security) to ensure confidentiality, integrity, authentication, and non-repudiation.

GOALS OF SOAP SECURITY

Goal	What It Ensures
Confidentiality	Message content is not accessible to unauthorized parties.
Integrity	Message content is not altered in transit.
Authentication	Verifies the identity of the sender and/or receiver.
Non-Repudiation	Provides cryptographic proof of origin and prevents denial of actions.

WS-SECURITY STANDARD

WS-Security is an OASIS standard that extends SOAP to add security tokens and related processing to SOAP messages -- independent of the transport layer.

COMMON SECURITY TOKENS

UsernameToken

SAML Token

X.509 Certificate

BinarySecurityToken

Always use HTTPS for transport security in addition to WS-Security -- message-level security and transport security solve different problems.

KEY TAKEAWAY SOAP security combines HTTPS transport with message-level WS-Security so identity survives intermediaries.

SOAP Security Overview

Securing SOAP Web Services for Confidentiality, Integrity, and Authentication



SOAP Security (WS-Security) is a standard specification that adds security to SOAP messages using XML-based security tokens and headers.

1. OVERVIEW



SOAP Security (WS-Security) provides a standard way to secure SOAP messages at the message level.

- Ensures Confidentiality
- Ensures Integrity
- Provides Authentication
- Prevents Replay Attacks
- Works with existing SOAP and XML infrastructure

2. SECURITY BENEFITS



Data Confidentiality
Protects data from being read by unauthorized parties.



Data Integrity
Ensures messages are not altered in transit.



Authentication
Verifies the identity of the sender.

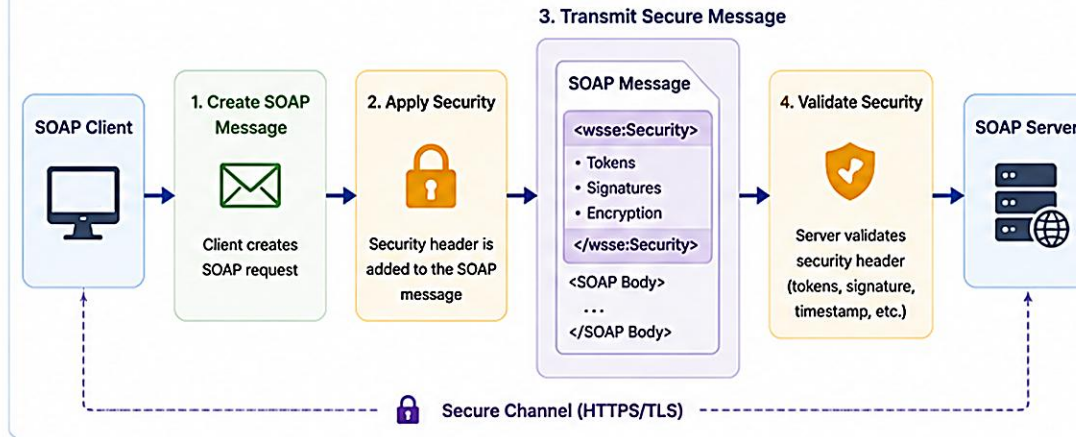


Non-Repudiation
Provides proof of message origin.

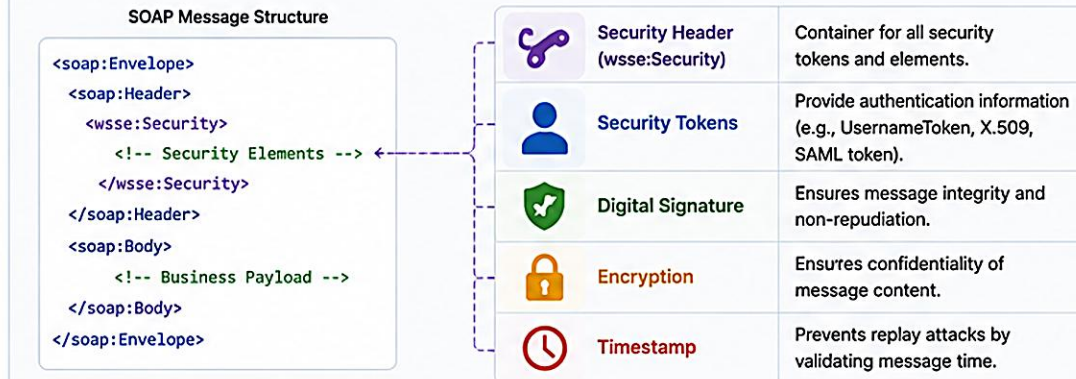
3. WS-SECURITY STANDARDS

- ☒ OASIS WS-Security 1.1 (2006)
- ☒ OASIS WS-Security 1.1.1 (UsernameToken Profile)
- ☒ OASIS WS-Security SAML Token Profile
- ☒ OASIS WS-Security X.509 Token Profile

4. HOW WS-SECURITY WORKS



5. WS-SECURITY MESSAGE ELEMENTS



6. COMMON SECURITY FEATURES



UsernameToken

Uses username/password credentials for authentication.



X.509 Certificate Token

Uses digital certificates for strong authentication.



SAML Token

Supports enterprise single sign-on (SSO) and federation.



Message Encryption

Encrypts SOAP body, headers, or specific elements.



Message Signature

Digitally signs whole message or specific parts.



Timestamp

Adds timestamp to ensure message freshness.



Non-Repudiation

Provides proof of message origin and integrity.

7. BEST PRACTICES



Always use HTTPS/TLS for transport security.



Use strong tokens (X.509 or SAML) for production systems.



Validate timestamps to prevent replay attacks.



Encrypt sensitive data inside the SOAP body.



Sign important parts of the message.



Keep security libraries and certificates up to date.

WS-SECURITY CONCEPTS

WS-Security extends SOAP messages by adding security information in the SOAP Header -- most commonly a UsernameToken.

wsse:Security Header (UsernameToken)

```
<soapenv:Header>
  <wsse:Security xmlns:wsse="...oasis-200401-wss-wssecurity-secext-1.0.xsd">
    <wsse:UsernameToken>
      <wsse:Username>crm-partner</wsse:Username>
      <wsse:Password Type="...PasswordText">
        lab24-shared-secret
      </wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soapenv:Header>
```

CORE WS-SECURITY CONCEPTS

- **Security Tokens** — carry authentication info about the sender (UsernameToken, SAML, X.509).
- **Digital Signature** — ensures integrity and authenticity using XML Signature.
- **Encryption** — protects message content using XML Encryption.
- **Timestamp** — prevents replay attacks by adding creation time and expiration.

HOW A WSS4J USERNAMETOKEN INTERCEPTOR WORKS



KEY TAKEAWAY Plaintext PasswordText UsernameToken is lab-only -- production needs TLS plus PasswordDigest (or better) and rotated secrets.

WS-Security in SOAP Web Services

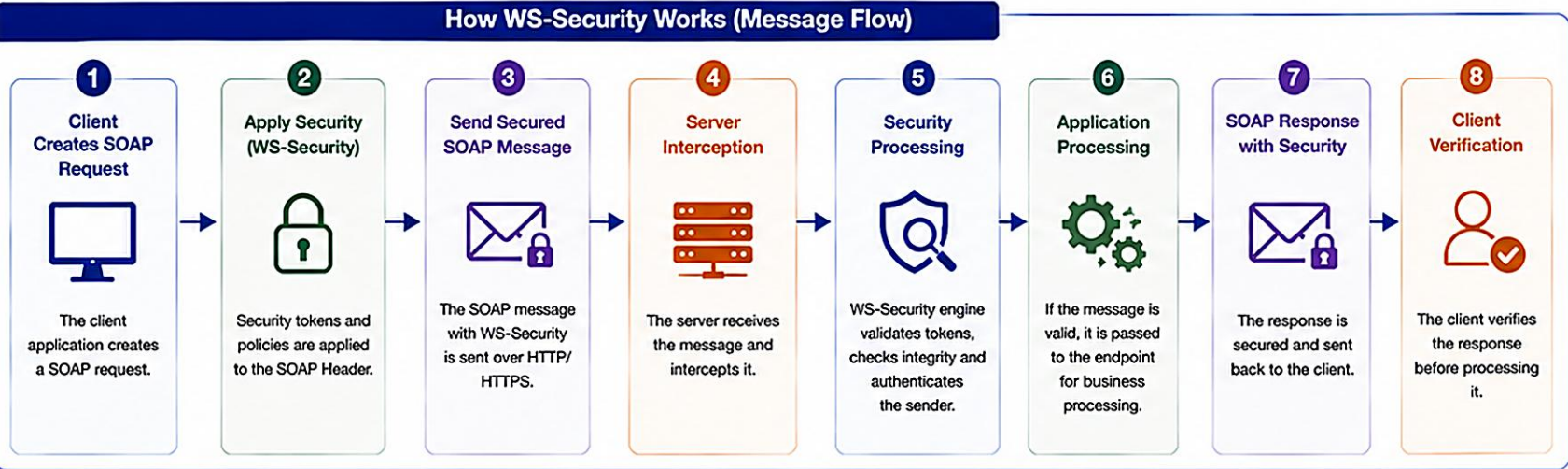
WS-Security is a standard (OASIS) that adds security to SOAP messages. It defines how to provide authentication, integrity, confidentiality and access control.

**Authentication**
Verify the identity of the sender

**Confidentiality**
Protect the message from eavesdropping

**Integrity**
Ensure the message is not tampered

**Access Control**
Ensure only authorized parties can act



WS-Security Header Structure

```
<soapenv:Header>
  <wsse:Security soapenv:mustUnderstand="1"
    xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/
    oasis-200401-wss-wssecurity-secext-1.0.xsd">
    <!-- Security Token -->
    <wsse:UsernameToken>
      <wsse:Username>john.doe</wsse:Username>
      <wsse:Password Type="....#PasswordDigest">...</wsse:Password>
      <wsse:Nonce>...</wsse:Nonce>
      <wsu:Created>2024-05-20T10:15:30Z</wsu:Created>
    </wsse:UsernameToken>
    <!-- Digital Signature -->
    <ds:Signature xmlns:ds="http://www.w3.org/2000/09/xmldsig#">...
    </ds:Signature>
    <!-- Encryption (optional) -->
    <xenc:EncryptedKey>...</xenc:EncryptedKey>
  </wsse:Security>
</soapenv:Header>

<soapenv:Body> ... </soapenv:Body>
```

Common WS-Security Elements

**Authentication**

- UsernameToken (Password / Digest)
- X.509 Certificate
- SAML Token
- Kerberos Token

**Integrity**

- XML Digital Signature
- Ensures message is not changed
- Signed parts: Body, Timestamp, etc.

**Confidentiality**

- XML Encryption
- Encrypts parts of the SOAP message
- Protects sensitive data in transit

**Non-Repudiation**

- Digital Signature
- Prevents sender from denying the message
- Protects proof of origin and integrity

**Timestamp**

- Prevents replay attacks
- <wsu:Timestamp> contains Created and Expires

Security Token Examples

**UsernameToken**
Used for simple authentication (user/password).

**X.509 Certificate**
Uses digital certificates for strong authentication.

**SAML Token**
Carries assertions about user identity and roles.

**Kerberos Token**
Enterprise authentication using Kerberos tickets.

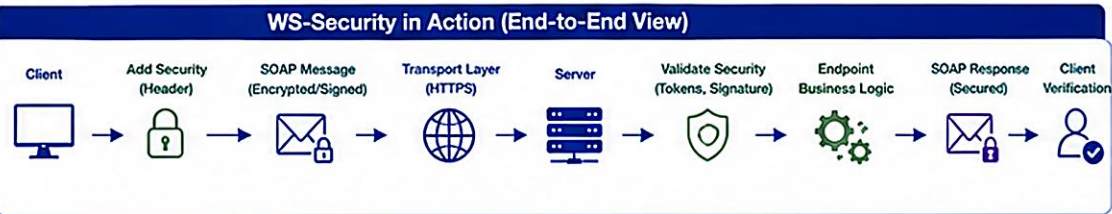
WS-Security Policy (Example)

```
<wsp:Policy>
  <sp:AsymmetricBinding>
    <wsp:Policy>
      <sp:InitiatorToken>
        <wsp:Policy>
          <sp:X509Token sp:IncludeToken="http://schemas.xmlsoap.org/
            ws/2005/07/securitypolicy/IncludeToken/AlwaysToRecipient"/>
        </wsp:Policy>
      </sp:InitiatorToken>
      <sp:RequireSignatureConfirmation/>
      <sp:Layout>
        <sp:LaxTimestampFirst/>
      </sp:Layout>
    </wsp:Policy>
  </sp:AsymmetricBinding>
</wsp:Policy>
```

Require X.509 certificate

Require signature confirmation

Include timestamp to prevent replay



Best Practices

- Always use HTTPS for transport security.
- Sign and encrypt sensitive SOAP messages.
- Use timestamps to prevent replay attacks.
- Validate all incoming tokens and signatures.
- Follow the principle of least privilege.



Key Takeaways

- WS-Security secures SOAP messages at the message level.
- It provides standards for authentication, integrity, confidentiality, and access control.
- It works independently of transport security (but HTTPS is recommended).
- It is widely used in enterprise environments and with legacy systems.

TRY IT YOURSELF — EXERCISE 5: USERNAMETOKEN PLAN

Reinforces: WSS4J UsernameToken validation you just saw -- planning the evidence, not implementing JWT.

STEP / WHAT TO DO

Step	What To Do
Goal	Outline UsernameToken evidence without implementing JWT.
File	notes/username-token-plan.md (in examples/module-24-exercises/).
Happy path	Secured GetCustomer for CUS-1001 (crm-partner / lab24-shared-secret) succeeds.
Failure path	Missing/invalid token produces a distinct security fault from a not-found fault.
Deliverable	Secret hygiene rule (never real prod passwords) stated; Bearer JWT explicitly deferred to Lab 28.
Reference	exercises/exercise-05-username-token-plan.md

The Two Scenarios You Are Planning For

```
requests/get-customer-secured.xml -> crm-partner / lab24-shared-secret UsernameToken -> Wss4j validates -> CustomerEndpoint runs
requests/get-customer.xml         -> no Security header                               -> Wss4j rejects -> SOAP security fault (never reaches endpoint)
```

TRY IT NOW Complete Exercise 5 before continuing -- see exercises/exercise-05-username-token-plan.md.

INTEGRATING SOAP SERVICES (1 OF 2)

Integrating SOAP services means consuming external SOAP web services in your application using standard SOAP messages.

WAYS TO INTEGRATE SOAP SERVICES

- **1. WSDL-First Approach** — use the service's published WSDL to generate client stubs and strongly typed Java classes.
- **2. Contract-First Approach** — define your own contract (WSDL/XSD) first, then generate client and server artifacts.
- **3. Dynamic / Template Approach** — use WebServiceTemplate for dynamic requests without generating stubs.

INTEGRATION FLOW (SOAP CLIENT)



KEY TAKEAWAY Integrating SOAP services ensures seamless, standards-compliant communication with external systems.

Integrating SOAP Services

Connect and consume external SOAP Web Services in enterprise applications



SOAP integration enables systems to communicate securely and reliably across organizational boundaries.

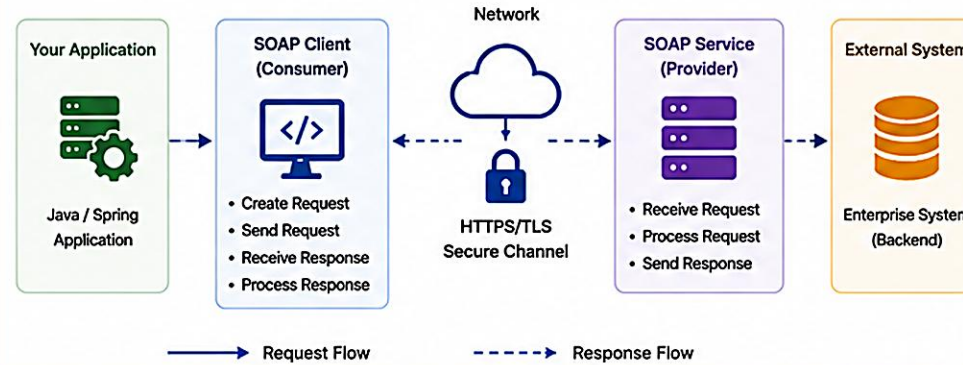
1. OVERVIEW



SOAP integration allows your application to communicate with external systems exposing SOAP web services.

- ✓ Standards-based (XML, WSDL)
- ✓ Platform independent
- ✓ Enterprise-grade security
- ✓ Reliable and transactional
- ✓ Widely used in legacy and enterprise systems

2. SOAP INTEGRATION ARCHITECTURE



3. INTEGRATION APPROACHES



Code-First (Client-First)

- Generate client from WSDL
- Use Spring WS / WebServiceTemplate
- Quick to consume existing services



Contract-First

- Define contract (WSDL/XSD) first
- Generate server stubs
- Build service implementation



Service to Service Integration

- System-to-system communication
- Orchestration and routing
- Error handling and monitoring

4. KEY CONSIDERATIONS



Security
Include-Security, SSL/TLS, and proper authentication.



WSDL & Versioning
Ensure compatibility and manage service versions.



Performance
Optimize with caching, connection pooling, and async calls.

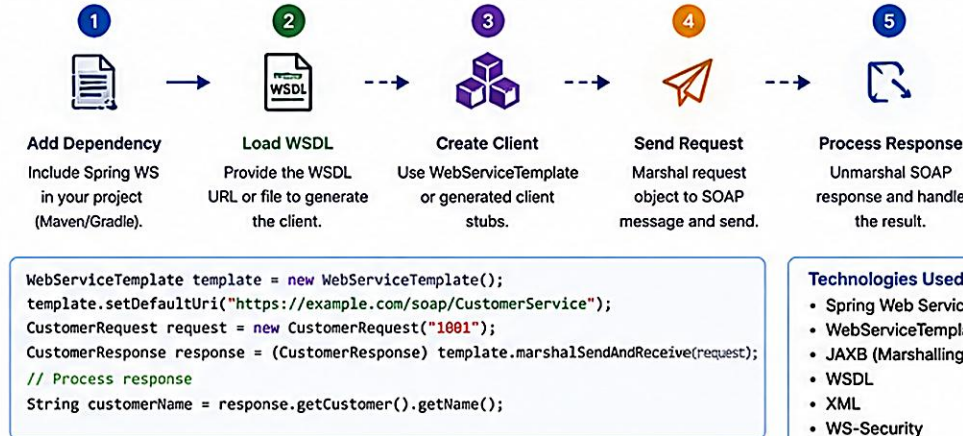


Error Handling
Handle SOAP faults and network errors gracefully.



Monitoring & Logging
Track requests, responses, and integration metrics.

5. EXAMPLE: CONSUMING A SOAP SERVICE IN SPRING



6. BENEFITS



Interoperability
Works across different platforms and technologies.



Reliability
Supports standards for security, transactions, and fault handling.



Security
Enterprise-grade security with WS-Security and SSL/TLS.



Reusability
Existing enterprise services can be consumed easily.



Loose Coupling
Systems remain independent and loosely coupled.

7. BEST PRACTICES



Always use HTTPS/TLS for secure communication.



Use WS-Security for authentication and message protection.



Implement timeouts and retry mechanisms.



Log requests and responses (with sensitive data masked).



Validate input and handle SOAP faults effectively.



Monitor and track integration performance and errors.



Keep WSDLs and dependencies version controlled.

INTEGRATING SOAP SERVICES (2 OF 2)

A sample Spring WS client, best practices, and how to handle SOAP faults from a downstream partner service.

Sample SOAP Client Code (Spring WS)

```
WebServiceTemplate template = new WebServiceTemplate();
template.setDefaultUri("https://partner.example.com/soap/CustomerService");

QName qname = new QName("http://northstar.com/crm/customer", "getCustomerRequest");
GetCustomerRequest request = new GetCustomerRequest();
request.setCustomerId("CUS-1001");
GetCustomerResponse response = (GetCustomerResponse) template.marshalSendAndReceive(qname, request);
```

BEST PRACTICES

- Always use HTTPS; validate XML with XSD before sending.
- Use WS-Security for authentication and integrity.
- Handle SOAP Faults and timeouts gracefully; cache WSDL locally to reduce latency.

HANDLING FAULTS FROM A PARTNER

- Always catch SoapFaultClientException around marshalSendAndReceive calls.
- Log the fault code, message, and correlation id -- never assume the call succeeded.
- Watch for WSDL changes, version drift, and large-payload latency in production.

KEY TAKEAWAY A resilient SOAP client treats faults, timeouts, and WSDL drift as first-class failure modes, not edge cases.

ENTERPRISE SOAP BEST PRACTICES (1 OF 2)

Following best practices ensures SOAP web services are secure, reliable, maintainable, and well-integrated.

#	Practice	What It Means
1	Use Contract-First Approach	Define WSDL and XSD first to establish a clear, stable contract.
2	Follow Naming Conventions	Use consistent, meaningful names for operations, messages, and types.
3	Keep Contracts Simple & Versioned	Avoid over-complicated schemas; version your services (e.g. /v1, /v2).
4	Validate with XSD	Always validate XML requests and responses against the schema.
5	Use WS-Security Standards	Implement authentication, integrity, and encryption using WS-Security.

KEY TAKEAWAY Contract discipline and schema validation are the foundation everything else -- security, reliability, observability -- builds on.

ENTERPRISE SOAP BEST PRACTICES (2 OF 2)

Apply these practices consistently to deliver reliable, maintainable, well-observed enterprise SOAP services.

#	Practice	What It Means
6	Enforce HTTPS	Always use SSL/TLS for transport security, in addition to WS-Security.
7	Handle Faults Gracefully	Use SOAP Faults to return meaningful, non-leaking error information.
8	Use Idempotent Operations	Design operations to be idempotent so retries are safe.
9	Centralize Logging & Correlation	Log requests, responses, and faults; include correlation IDs for tracing.
10	Monitor, Alert & Document	Track service health, latency, and errors; keep contracts documented.

KEY TAKEAWAY Well-designed, secure, and observable SOAP services ensure long-term success in enterprise integration.

TRY IT YOURSELF — EXERCISE 6: LAB 24 READINESS CHECKLIST

Capstone check -- confirm the Lab 23 REST baseline is understood before you open the Spring-WS lab.

STEP / WHAT TO DO

Step	What To Do
Goal	Verify the Lab 23 REST baseline is understood before starting the Spring-WS lab.
File	notes/lab24-readiness.md (in examples/module-24-exercises/).
Dependency	Note that Lab 23's Boot app + CustomerService is required before Lab 24 begins.
Path	Write the copy target: examples/lab24-crm.
Evidence correlation	Record lab24-001 for SOAP evidence (REST may still use lab-request-001).
Deliverable	Readiness note ties SOAP to the Boot baseline; list Kafka and React as Week 4, not Lab 24.

Readiness Checklist (notes/lab24-readiness.md)

```
[ ] Lab 23 Boot app + CustomerService exist and run      [ ] Copy target confirmed: examples/lab24-crm
[ ] Correlation ids noted: lab24-001 (SOAP), lab-request-001 (REST)  [ ] Kafka + React deferred to Week 4
```

TRY IT NOW Complete Exercise 6 before continuing -- see exercises/exercise-06-lab24-readiness.md.

LAB OVERVIEW -- SOAP WEB SERVICE ENDPOINTS

Lab 24: SOAP Web Service Endpoints — Northstar CRM Spring-WS

BUSINESS SCENARIO

- A regional billing partner only integrates via SOAP/XML and must create, get, update status, and list customers using the Lab 13 contract.
- Ship Spring-WS beside the Lab 23 REST API: live WSDL from XSD, four operations, CLIENT faults for not-found/duplicate, UsernameToken required.
- Protocol may differ; business rules must not -- both protocols delegate to the same CustomerService.

LEARNING OBJECTIVES

- Explain contract-first SOAP and why the XSD -- not Java -- is the source of truth.
- Author an XSD and let Spring-WS generate WSDL dynamically from it.
- Implement @Endpoint with @PayloadRoot / @RequestPayload / @ResponsePayload.
- Translate BusinessException subtypes into structured SOAP faults.
- Configure a minimal WS-Security UsernameToken interceptor.

WHAT YOU BUILD

- Copy lab23-crm to lab24-crm; add Spring-WS + jaxb2 plugin; author customer.xsd; configure MessageDispatcherServlet + DefaultWsd11Definition; implement CustomerEndpoint with four @PayloadRoot operations; add SOAP fault mapping and a UsernameToken interceptor; verify with curl + MockWebServiceImpl; evidence with correlation lab24-001.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE), CUS-1002 (Ravi Singh, PROSPECT), and CUS-9999 (not-found) anchor every example.

LAB TASKS AND DELIVERABLES

Northstar CRM Spring-WS -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch Lab 23 and add Spring-WS + jaxb2-maven-plugin dependencies.
2	Author customer.xsd and generate JAXB request/response classes.
3	Configure MessageDispatcherServlet and a live /ws/customer.wsdl.
4	Map JAXB types and implement CustomerEndpoint's four operations.
5	Share exceptions and map SOAP faults (CLIENT for not-found/duplicate).
6	Add a Wss4jSecurityInterceptor for UsernameToken (WS-Security).
7	Prove REST and SOAP share one CustomerService (same data, both protocols).
8	Automate with MockWebServiceClient tests + a partner evidence pack.

EXPECTED DELIVERABLES

- CustomerEndpoint with four SOAP operations + customer.xsd / live WSDL.
- JAXB generation + CustomerSoapMapper; SOAP fault mapping.
- Working UsernameToken interceptor (lab secret only).
- requests/ sample XML + fault cases; CustomerEndpointTest green twice.
- Evidence that REST and SOAP share CustomerService; README runbook.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security 10 · Docs 10

KEY TAKEAWAY Duplicating business rules or skipping WS-Security both lose core marks -- committing real secrets is an honor violation.

MODULE 24 SUMMARY

SOAP Web Service Endpoints and Request Mapping -- what we covered and how it fits together.

KEY CONCEPTS COVERED



Contract-First & Recap

SOAP architecture, WSDL, and XML Schema as the source of truth.



Endpoints & Mapping

@Endpoint and @PayloadRoot route SOAP requests to Java methods.



Lifecycle & Marshalling

Request/response lifecycle and XML marshalling/unmarshalling.



SOAP Faults

Structured faultcode/faultstring errors instead of stack traces.



WS-Security

UsernameToken and message-level security concepts.



Enterprise Best Practices

Contracts, security, reliability, and observability for SOAP.

MODULE FLOW RECAP

1

Contract (WSDL/XSD)

2

@Endpoint/@PayloadRoot

3

Marshalling

4

Faults

5

WS-Security

KEY TAKEAWAY You can now design, build, secure, and integrate production-ready SOAP endpoints with Spring WS -- contract-first, end to end.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of SOAP fundamentals, WSDL, and Spring WS endpoint annotations.

1. What is the role of WSDL in a SOAP web service?

- A. To describe the service implementation
- B. To define the service contract, operations, and messages**
- C. To handle service security
- D. To store service logs

3. Which annotation maps an incoming request to a specific XML namespace and local part?

- A. @RequestMapping
- B. @PayloadRoot**
- C. @Namespace
- D. @XmlElement

2. Which annotation is used to define a SOAP endpoint in Spring WS?

- A. @Service
- B. @Endpoint**
- C. @WebService
- D. @Component

4. What is the purpose of XML Marshalling in Spring WS?

- A. Convert XML to Java objects
- B. Convert Java objects to XML**
- C. Validate XML against XSD
- D. Encrypt XML messages

KEY TAKEAWAY WSDL, @Endpoint, and @PayloadRoot are the foundation of every Spring WS contract-first service.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on unmarshalling, WS-Security, contract-first design, and integration.

5. Which of the following is TRUE about XML Unmarshalling?

- A. It converts Java objects to XML
- B. It converts XML into Java objects**
- C. It is used only for request messages
- D. It is not required in Spring WS

7. Which approach is recommended for enterprise SOAP service development?

- A. Code-First approach
- B. Contract-First approach**
- C. Database-First approach
- D. Manual-First approach

6. WS-Security primarily provides which capabilities?

- A. Compression of SOAP messages
- B. Authentication, integrity, and confidentiality**
- C. Message routing and queuing
- D. Database encryption

8. What is a key benefit of integrating SOAP services with WebServiceTemplate?

- A. Faster UI rendering
- B. Loose coupling and reuse across systems**
- C. Elimination of databases
- D. Replacing REST APIs

KEY TAKEAWAY Contract-first design and WS-Security are what make a SOAP integration enterprise-ready.

KNOWLEDGE CHECK (3 OF 3)

Four scenario questions on SOAP faults, UsernameToken security, and shared business rules.

9. A CustomerNotFoundException is thrown inside CustomerEndpoint. What should the client receive?

- A. An HTTP 500 with a full Java stack trace
- B. A SOAP Fault with faultcode Client and a clear faultstring**
- C. A silent empty response
- D. The connection is simply closed

11. Why must CustomerEndpoint delegate to CustomerService instead of re-implementing business rules?

- A. SOAP endpoints cannot call service classes
- B. So REST and SOAP never diverge on business rules**
- C. It makes the WSDL generate faster
- D. JAXB requires it

10. True or False: A request without a valid UsernameToken should be rejected before business logic runs.

- A. True — enforced before the endpoint method runs**
- B. False

12. True or False: HTTPS alone is a complete substitute for WS-Security UsernameToken.

- A. True
- B. False — transport security and message-level security solve different problems**

KEY TAKEAWAY SOAP faults, WS-Security, and a single shared service are the three pillars of a trustworthy enterprise SOAP endpoint.

MODULE 24 COMPLETE

SOAP Web Service Endpoints and Request Mapping

You can now design, build, secure, and integrate contract-first SOAP endpoints with Spring WS -- @Endpoint, @PayloadRoot, marshalling, faults, and WS-Security.